

基于混合智能算法的烟幕干扰弹投放策略

摘 要

烟幕干扰弹是借助化学爆炸或燃烧分散汇聚成气溶胶云团或烟幕，可遮蔽目标前方的特定空域，达到干扰敌方导弹的目的。其通过多种方式由无人机进行投放，从而实现烟幕干扰弹的精准抛掷，并可借助时间引信时序控制它的起爆时间。

针对问题一，通过分别建立导弹、无人机和烟幕云团的**刚体运动模型**，描述导弹 M1、无人机 FY1 和烟幕云团的运动过程中的实时位置变化，然后建立烟幕干扰弹对真目标的**有效遮蔽判定模型**，并基于此模型求解烟幕干扰弹对真目标的有效遮蔽时间为 $1.391s$ 。

针对问题二，在问题一的基础上延续**有效遮蔽判定模型**，通过构建以无人机飞行方向、飞行速度、烟幕干扰弹投放时间及起爆时间作为决策变量的**单机单目标优化模型**，以烟幕干扰弹对导弹 M1 的有效遮蔽时长作为目标函数，使用**分布式模拟退火^[1]-粒子群混合算法**，求解烟幕干扰弹对真目标的有效遮蔽时间为 $4.576s$ 。

针对问题三，需满足单机 FY1 连续投掷三枚烟幕弹干扰导弹 M1，本问题在问题二单弹优化模型的基础上，通过引入多弹协同时序控制机制，以三弹累积有效遮蔽时长为优化目标，构建**进一步优化的单机单目标优化模型**，采用模拟退火-粒子群整合算法优化优化无人机运动轨迹与弹群动作序列，实现了三弹遮蔽时段的延续，最终在该模型上解得烟幕干扰弹对真目标的有效遮蔽时间为 $6.470s$ ，详细策略见附件 *result1.xlsx*。

针对问题四，在**进一步优化的单机单目标优化模型**的基础上，创新构建多机独立决策框架，通过联合优化多机的飞行方向、速度以及各无人机独立的烟幕弹投放时间与起爆时间，以三弹非重叠遮蔽总时长为目标函数，构建**多机单目标优化模型**，采用**分布式模拟退火-粒子群混合算法**实现多机运动协同与遮蔽时段精准衔接，使有效遮蔽时长达到 $11.430s$ ，并通过 *result2.xlsx* 输出投放策略。

针对问题五，在前四问的基础上，创新构建涵盖无人机飞行参数、多弹投放起爆时序及弹目动态匹配关系的**高维多目标优化模型**，以三枚导弹总有效遮蔽时长为优化目标，在严格满足单机载弹量上限、弹间动作异步性、烟幕时空有效性等复杂约束下，采用分层式模拟退火-粒子群混合算法实现任务分配与时空轨迹的全局协同优化，最终得到满足所有条件的最大有效遮蔽时间为 $22.850s$ ，详细策略见附件 *result3.xlsx*。

本文通过构建多层次时空协同优化模型，结合混合智能算法的全局寻优能力，解决了从单机单弹定时干扰到多机多弹协同防御的系列决策难题，显著提升了烟幕遮蔽时长并输出了完备作战方案，为复杂的真实环境下无人机集群干扰策略提供了理论支撑。

关键词：有效遮蔽判定模型 模拟退火-粒子群混合算法 高维多目标优化模型

一、问题重述

1.1 问题背景



图 1 无人机投放示意图

烟幕干扰弹是借助化学爆炸或燃烧分散汇聚成气溶胶云团或烟幕，可遮蔽目标前方的特定空域，达到干扰敌方导弹的目的，且其具备效费比高，成本低等优点（如上图所示）。烟幕干扰弹可通过多种方式由无人机进行投放，从而实现烟幕干扰弹的精准抛掷，并可借助时间引信时序控制它的起爆时间。

1.2 问题的提出

现由具备长时间续航能力的无人机搭载着某型号的烟幕干扰弹在特定区域巡飞，在接受到任务后，无人机在来袭武器与需要保护的目标之间投放烟幕干扰弹形成有效的烟幕遮蔽。现需建立数学模型，解决下列问题：

问题一：无人机 FY_1 投掷 1 枚烟幕干扰弹实现对导弹 M_1 的干扰，无人机 FY_1 速度以 $120m/s$ 等高度的朝假目标方向移动，在接收任务 $1.5s$ 后投掷，投掷后经过 $3.6s$ 起爆，求此过程中烟幕干扰弹对导弹 M_1 的有效遮蔽时长。

问题二：在无人机 FY_1 投掷 1 枚烟幕干扰弹实现对导弹 M_1 的干扰过程中，要求确定无人机飞行的方向、飞行速度、烟幕干扰弹的投放点以及烟幕干扰弹的起爆点，使烟幕干扰弹的遮蔽时间最大化。

问题三：要求无人机 FY_1 投放 3 枚烟幕干扰弹实现对导弹 M_1 的干扰，要求确定无人机飞行的方向、飞行速度、烟幕干扰弹的投放点以及烟幕干扰弹的起爆点，使烟幕干扰弹的遮蔽时间最大化，以此实现最优的烟幕干扰弹投放策略。

问题四：三架无人机 FY_1 、 FY_2 、 FY_3 各投掷 1 枚烟幕干扰弹实现对导弹 M_1 的干扰，要求确定无人机飞行的方向、飞行速度、烟幕干扰弹的投放点以及烟幕干扰弹的起爆点，使烟幕干扰弹的遮蔽时间最大化，以此实现最优的烟幕干扰弹投放策略。

问题五：需利用五架无人机且每架无人机最多投掷三枚烟幕干扰弹，实现对导弹 M_1 、 M_2 、 M_3 的干扰，要求确定无人机飞行的方向、飞行速度、烟幕干扰弹的投放点以及烟幕干扰弹的起爆点，使烟幕干扰弹的遮蔽时间最大化，以此实现最优的烟幕干扰弹投放策略。

二、问题分析

2.1 问题一的分析

针对固定飞行策略下的单个烟雾干扰弹的有效遮蔽时长计算问题，问题一给定无人机 FY_1 以 120m/s 速度朝假目标飞行、接收任务 1.5 秒后投掷烟幕干扰弹、 3.6 秒后起爆的约束条件下，通过建立刚体运动学模型刻画了导弹和无人机的匀速直线运动、只受重力作用的烟雾干扰弹做抛物线运动（水平方向上初始速度等于无人机的水平速度）以及起爆后云团以 3m/s 速度做垂直匀速下沉的三种运动，根据题目定义的 10m 的空间硬性阈值（云团中心在 10m 范围内达到有效遮蔽）和 20s 的时间窗口（即起爆后 20s 云团消散）构建有效遮蔽判定模型，最终得到导弹的轨迹与云团下降轨迹的空间距离方程在时间上达到双重的有效性约束（即此时的空间距离 $\leq 10\text{m}$ 且时间 $\in [\text{起爆时刻}, \text{起爆时刻}+20\text{s}]$ ）的连续区间，进而输出固定的有效遮蔽时长。

2.2 问题二的分析

针对单个无人机有效遮蔽时长的最大化问题，问题二需要通过构建以无人机飞行方向、飞行速度、烟幕干扰弹投放时间及起爆时间作为决策变量的单一目标优化模型，在满足无人机同等高度匀速直线飞行、烟幕干扰弹投放与起爆的时间间隔大于等于极限条件下的自由落体等约束条件下，以烟幕干扰弹对导弹 $M1$ 的有效遮蔽时长作为目标函数；然后使用模拟退火算法^[2]和粒子群算法相结合的混合算法，利用粒子群算法实现快速定位，得出更精确的解，而后通过模拟退火算法的特殊机制跳出局部最优陷阱，在空间中整体探寻最优的组合形式，最终输出能使云团有效遮蔽时长最大化的无人机航向、速度、干扰弹的投弹点以及起爆点。

2.3 问题三的分析

针对单机三弹协同干扰问题，问题三在无人机 FY_1 匀速直线飞行且保持等高度的前提下，通过扩大决策变量体系（此决策体系包含了飞行方向、飞行速度，三枚烟幕弹各自的投放时间与起爆时间），构建以三枚干扰弹累积有效遮蔽时长作为目标函数的单目标优化模型，且需满足烟幕干扰弹起爆后 20 秒时效性约束和相邻两弹间投放时间间隔需大于等于 1 秒的时间约束条件；然后采用模拟退火算法与粒子群算法结合的优化策略，利用粒子群算法迅速生成多干扰弹协同调度方案，并通过模拟退火算法突破局部最优解，最终在飞行参数与多弹时序的优化中实现遮蔽时长最大化，同时输出全局的无人机运动策略及三弹投放和起爆时序表。

2.4 问题四的分析

针对三机协同单弹干扰问题，问题四需要在三架无人机独立操控且每机仅投放一枚烟幕干扰弹的前提下，通过构建九维的决策变量体系（决策变量包含三架无人机的飞行方向、飞行速度及干扰弹独立投放时间与起爆时间），建立以三弹累积遮蔽时长为优化目标的协同模型，同时满足单机匀速飞行且等高度、弹间投放无时间冲突以及遮蔽有效窗口等约束条件；进而采用改进的模拟退火-粒子群整合优化架构，利用粒子群算法的搜寻三机飞行参数组合，并通过模拟退火机制规避航迹的冲突或遮蔽时段冗

余，最终在整体优化中实现三枚烟幕弹遮蔽效果的连续性，输出总遮蔽时长最大化的协同飞行方案和起爆时刻。

2.5 问题五的分析

针对五机多弹多目标协同干扰问题，问题五需在调度五架无人机对三枚来袭导弹完成最优的遮蔽策略规划，通过构建高维决策体系（决策体系包含每架无人机的飞行方向、飞行速度、干扰弹投放时序与起爆时间），建立以三枚导弹总有效遮蔽时长为优化目标的模型，且需满足单机匀速飞行且等高度、烟幕干扰弹投放时间间隔大于等于1秒及遮蔽时空有效性等约束条件；接着采用模拟退火-粒子群整合优化模型，首先利用粒子群算法快速生成无人机与导弹任务分组，并生成最初的弹量分配策略，再通过模拟退火机制优化弹群时序与飞行参数，最终在空间中实现无人机与干扰弹的调度与时空协同的整体最优解，输出此解下的三目标遮蔽总时长最大化的完整作战策略。

三、模型假设

1. 假设无人机在进行路线选取时，有充分的燃料，无须担心无人机的续航问题。
2. 假设烟幕干扰弹被投放后，水平方向的速度即为无人机的速度
3. 假设烟幕云团不受空气阻力和环境变化等外在条件的影响
4. 假设各运动体独立运动，彼此互不影响运动状态。

四、符号说明

符号	说明	单位
$r_{FY_j}(t)$	无人机 FY_j 在任意时刻 t 的位置	m
$p_{j,m}^{fall}$	瞬间释放烟幕干扰弹的投放点位置	m
$C_{j,m}(t)$	云团球心在任意时刻 t 的位置	m
r_{cloud}	单枚烟幕云团的半径	m
$t_{j,m}^{delay}$	释放烟幕干扰弹后的引信延迟时间	s
$t_{j,m}^{fall}$	无人机 FY_j 在受领任务时间	s
$t_{j,m}^{blow}$	烟幕干扰弹的起爆时间	s
$T_{j,m}$	单枚烟幕云团的有效遮蔽时长	s
$d_{Mi,c}$	导弹到烟幕云团球心的距离	m
$\varphi_i(t)$	真目标圆柱体的可见轮廓点集	m

五、模型的建立与求解

5.1 问题一模型的建立与求解

问题一首先通过分别建立导弹、无人机和烟幕云团的刚体运动模型，描述导弹 M_1 、无人机 FY_1 和烟幕云团的运动过程中的实时位置变化，然后建立烟幕干扰弹对真目标的有效遮蔽判定模型，并基于此模型求解烟幕干扰弹对真目标的有效遮蔽时间。

5.1.1 坐标系与单位的确立

根据题意，采用空间直角坐标系，以假目标为原点 $O = (x_0, y_0, z_0) = (0, 0, 0)$ ，真目标为一圆柱体， y 轴正方向指向其下底面圆心 $Q_c = (x_c, y_c, z_c) = (0, 200, 0)$ ， z 轴正方向竖直向上， x 轴正方向由右手定则定理确定。

真目标圆柱体的半径为 $r_c = 7m$ ，高为 $h = 10m$ ，轴线与 z 轴相互平行。无人机运动过程中的航向角 θ 在 xOy 平面内自 x 轴正方向起逆时针旋转，即：

$$\theta = (\cos(\theta), \sin(\theta), 0) \quad (1)$$

5.1.2 导弹刚体运动模型的建立

根据假设 1，导弹 M_i 运动过程中的飞行速度恒定为 $v_M = 300m/s$ ，飞行方向是从导弹的初始位置 $r_{Mi}(0) = (x_{Mi}, y_{Mi}, z_{Mi})$ 朝向假目标 $O = (x_0, y_0, z_0) = (0, 0, 0)$ ，可通过计算二者之间的方向向量表示：

$$\overrightarrow{r_{Mi, O}} = (x_0 - x_{Mi}, y_0 - y_{Mi}, z_0 - z_{Mi}) \quad (2)$$

对方向向量 $\overrightarrow{r_{Mi, O}}$ 单位化，可得导弹 M_i 在运动方向上的单位向量 $\hat{v}_{Mi}$ ：

$$\hat{v}_{Mi} = \frac{\overrightarrow{r_{Mi, O}}}{|\overrightarrow{r_{Mi, O}}|} \quad (3)$$

其中， $|\overrightarrow{r_{Mi, O}}|$ 表示导弹 M_i 的初始位置到假目标的模长，即：

$$|\overrightarrow{r_{Mi, O}}| = \sqrt{(x_0 - x_{Mi})^2 + (y_0 - y_{Mi})^2 + (z_0 - z_{Mi})^2} \quad (4)$$

随时间 t 的变化，导弹 M_i 的飞行路径可表示为：

$$r_{Mi}(t) = r_{Mi}(0) + v_M \cdot \hat{v}_{Mi} \cdot t \quad (5)$$

导弹 M_i 在任意时刻 t 的位置可表示为 $|x_{Mi}(t)|$ 、 $|y_{Mi}(t)|$ 、 $|z_{Mi}(t)|$ 。

5.1.3 无人机和烟幕云团的运动模型的建立

根据题目条件可知，无人机 FY_j 的飞行路径属于匀速直线运动，且高度保持不变。其中初始位置为 $r_{FYj}(0) = (x_{FYj}, y_{FYj}, z_{FYj})$ ，飞行角度 $\theta (\theta \in [0, 2\pi])$ ，即飞行方向的单位向量 $\hat{v}_{FYj}$ 为：

$$\hat{v}_{FYj} = (\cos(\theta), \sin(\theta), 0) \quad (6)$$

由于无人机 FY_j 的飞行高度保持不变, 因此 $\hat{v}_{FY_j}$ 只在水平面上有分量。

在飞行过程中, 无人机 FY_j 的飞行速度为 v_{FY_j} ($v_{FY_j} \in [70, 140] m/s$), 其在任意时刻 t 的位置 $r_{FY_j}(t)$ 可表示为:

$$r_{FY_j}(t) = r_{FY_j}(0) + v_{FY_j} \cdot \hat{v}_{FY_j} \cdot t \quad (7)$$

由此可得, 无人机 FY_j 在任意时刻 t 的位置受飞行速度 v_{FY_j} , 飞行方向的向量 $\hat{v}_{FY_j}$ 的影响。无人机 FY_j 在受领任务 $t_{j,m}^{fall}$ 秒后, 瞬间释放第 m 枚烟幕干扰弹, 此时投放位置为 $r_{FY_j}(t_{j,m}^{fall})$, 即瞬间释放烟幕干扰弹的投放点 $p_{j,m}^{fall}$ 为:

$$p_{j,m}(0) = r_{FY_j}(t_{j,m}^{fall}) = (x_{j,0}, y_{j,0}, z_{j,0}) \quad (8)$$

由假设 2 可知, 投放瞬间烟雾干扰弹在水平方向上的速度等于无人机 FY_j 的飞行速度 v_{FY_j} , 且在竖直方向上只受重力作用, 整体运动轨迹如下图所示:

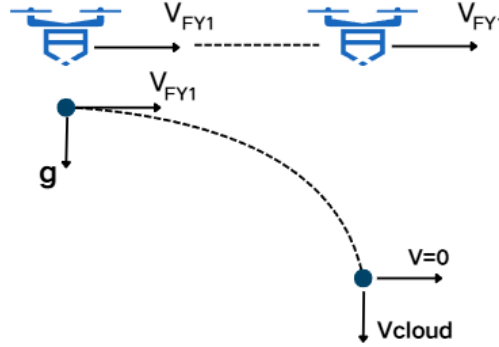


图 2 烟雾干扰弹的运动轨迹示意图

烟幕干扰弹在投放后的任意时刻 t (其中 $t \geq t_{j,m}^{fall}$)的位置可表示为:

$$P_{j,m}(t) = p_{j,m}(0) + (t - t_{j,m}^{fall}) \cdot v_{FY_j} \cdot \hat{v}_{FY_j} - \frac{1}{2} \cdot g \cdot (t - t_{j,m}^{fall})^2 \cdot \vec{e}_z \quad (9)$$

其中, $\vec{e}_z = (0, 0, 1)$ 表示沿竖直方向向上的单位向量, g 表示重力加速度, 取 $g = 9.8 m/s^2$ 。

无人机受领任务后, 假设经过 $t_{j,m}^{delay}$ 秒后, 烟幕干扰弹起爆, 即从无人机 FY_j 开始运动到烟幕干扰弹起爆的总时间 $t_{j,m}^{blow}$ 为:

$$t_{j,m}^{blow} = t_{j,m}^{fall} + t_{j,m}^{delay} \quad (10)$$

则烟幕干扰弹的位置可表示为 $P_{j,m}(t_{j,m}^{blow})$ 。由题目条件可知, 烟幕干扰弹在起爆瞬间可形成球状烟幕云团, 且以固定速度 $v_{cloud} = 3 m/s$ 下沉, 云团球心在任意时刻 t (且 $t \geq t_{j,m}^{blow}$) 的位置 $C_{j,m}(t)$ 可表示为:

$$C_{j,m}(t) = P_{j,m}(t_{j,m}^{blow}) - v_{cloud} \cdot (t - t_{j,m}^{blow}) \cdot \vec{e}_z \quad (11)$$

烟幕云团仅在起爆20s内可为目标提供有效遮蔽, 故上式中 $t \in [t_{j,m}^{blow}, t_{j,m}^{blow} + 20]$ 。

5.1.4 有效遮蔽判定模型的建立

根据题目条件,烟幕云团的本质作用是为目标提供有效遮蔽。对于某一导弹 M_i 在某一时刻 t ,通过判断该时刻导弹和目标之间的视线是否被烟幕云团阻断,定义该时刻是否为“有效遮蔽”的布尔判定为:

$$I_i(t) \in \{0, 1\} \quad (12)$$

通过这一基本思想,可将有效遮蔽判定模型分为以下两个部分讨论:

1. 导弹在云团内部判定

当导弹落入烟幕云团的内部时,导弹和目标之间的视线被烟幕云团阻断,为有效遮蔽,即:

$$d_{M_i,c} = \|r_{M_i}(t) - C_{j,m}(t)\| \leq R_{cloud} \quad (13)$$

其中, $d_{M_i,c}$ 表示导弹 M_i 到烟幕云团球心的距离, $R_{cloud} = 10m$ 表示球状烟幕云团的半径。此时 $I_i(t) = 1$ 。

2. 导弹在云团外部判定

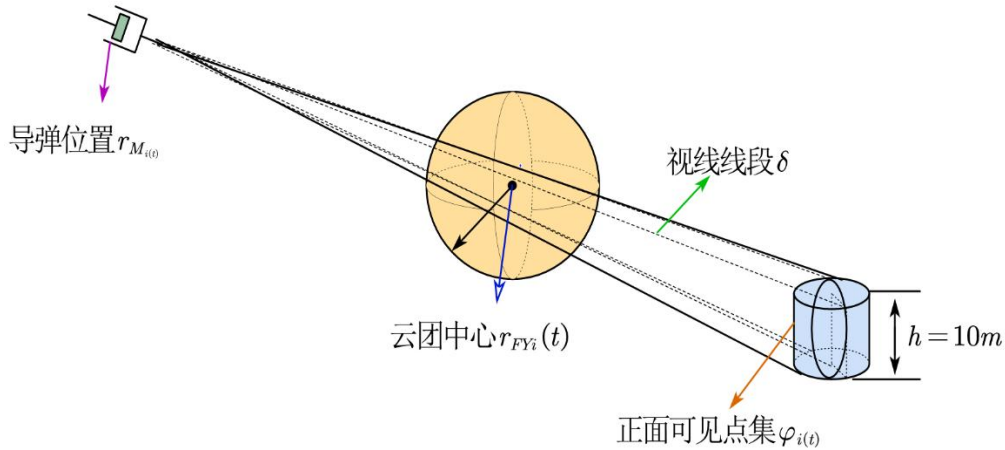


图 3 导弹位于烟幕云团外部时的示意图

如上图所示,当导弹位于云团外部时,须判断导弹到真目标外侧的所有可见点是否被烟幕云团阻断。根据题目条件可知,真目标为一圆柱体,半径 $r_c = 7m$,高 $h = 10m$,下底面圆心为 $Q_c = (x_c, y_c, z_c) = (0, 200, 0)$,则圆柱体外表面集合为:

$$\Phi_c = \{(x, y, z) \mid (x^2 + y^2)^{1/2} = r_c, z \in [z_0, z_0 + h]\} \quad (14)$$

其中, z_0 表示真目标下底面的 z 轴坐标。

在导弹的运动过程中,真目标对其存在“可见面”和“背面”。通过视线判定,可以筛选出正面可见点集合,避免将背面不可见点作为需要遮蔽的对象。即当“从圆柱体外表面集合中任意点 p 指向导弹的方向”和“在该点 p 的外法向方向”相同时,该点 p 才为正面可见点。其中点 p 的外侧法向量可表示为:

$$n(p) = \left(\frac{x - x_c}{r_c}, \frac{y - y_c}{r_c}, 0 \right) \quad (15)$$

则正面可见点集合 $\varphi_i(t)$ 为:

$$\varphi_i(t) = \{p \mid (M_i(t) - p) \cdot n(p) > 0\} \quad (16)$$

对于每一个正面可见点, 导弹 $M_i(t)$ 到点 p 的视线线段 $\delta(a)$ 为:

$$\delta(a) = M_i(t) + a(p - M_i(t)), \quad a \in [0, 1] \quad (17)$$

设视线线段 δ 的方向向量为 $\hat{v}_{Mp}$, 从导弹 M_i 在任意时刻的位置 $M_i(t)$ 出发, 沿 $\hat{v}_{Mp}$ 的有向直线参数化:

$$\gamma(b) = M_i(t) + b\hat{v}_{Mp}, \quad b \in R \quad (18)$$

烟幕云团的球心到该直线的最近距离为:

$$d_c = d_{Mi,c} \cdot \sin \beta \quad (19)$$

其中, $d_{Mi,c}$ 表示导弹 M_i 到烟幕云团球心的距离, β 表示从导弹 M_i 位置 $M_i(t)$ 出发指向球心和指向正面可见点集合中的点 p 两个方向的夹角, 即:

$$\hat{v}_c = \frac{C_{j,m}(t) - M_i(t)}{\|C_{j,m}(t) - M_i(t)\|} \quad \hat{v}_{Mp} = \frac{p - M_i(t)}{\|p - M_i(t)\|} \quad (20)$$

$$\cos \beta = \hat{v}_c \cdot \hat{v}_{Mp}, \quad \beta = \arccos(\hat{v}_c \cdot \hat{v}_{Mp}), \quad \beta \in [0, \pi] \quad (21)$$

其中, $\hat{v}_c$ 表示从导弹 M_i 位置 $M_i(t)$ 出发指向球心的方向, $\hat{v}_{Mp}$ 从导弹 M_i 位置 $M_i(t)$ 出发指向正面可见点集合中的点 p 的方向。

而球心在该直线上的投影参数为:

$$b_c = d_{Mi,c} \cdot \cos \beta \quad (22)$$

当 $d_c > R$ 时, 视线线段与烟幕云团不相交; 当 $d_c \leq R$ 时, 视线线段与烟幕云团相交。当对所有正面可见点集合 $\varphi_i(t)$ 评估的点 p 都成立, 则满足“有效遮蔽”的布尔判定, 记 $I_i(t) = 1$ 。

对于单枚烟幕云团, 其有效遮蔽时长可表示为:

$$T_{j,m} = \int_{t_{j,m}^{blow}}^{t_{j,m}^{blow} + 20} I_i(t) dt \quad (23)$$

其中, $t_{j,m}^{blow}$ 表示烟幕干扰弹在起爆瞬间可形成球状烟幕云团的时间。

5.1.5 问题一有效遮蔽时长的求解

根据题目条件可知, 问题一明确规定导弹 M_1 的初始位置 $r_{M_1} = (20000, 0, 2000)$,

无人机 FY_1 的初始位置 $r_{FY_1} = (17800, 0, 1800)$ 、飞行速度 $v_{FY_1} = 120m/s$ 、飞行方向向量 $\hat{v}_{FY_1} = (-1, 0, 0)$ 、受领任务时间 $t_{1,1}^{fall} = 1.5s$ ，以及投放后间隔 $t_{1,1}^{delay} = 3.6s$ 秒后引爆第1枚烟幕干扰弹。

通过建立的有效遮蔽判定模型，具体求解过程如下：

烟幕干扰弹对 M_1 的有效遮蔽时长求解：

Step 1: 初始化变量

建立时间轴，设定固定变量、数值容差、时间步长等。

Step 2: 变量计算

给出投放点、起爆中心和导弹位置等变量的计算公式。

Step 3: 圆柱轮廓采样

从角向和竖向生成侧面点集，通过判断逻辑只保留前向可见的点。

Step 4: 沿时间轴逐点判定 A/B

A 判据：若导弹在云内，直接计时；

B 判据：反之，判定是否满足前向性、角度覆盖和线段穿球。

Step 5: 结果计算

统计 A 与 B 的连续时间段及总时长，总有效时间满足 A 与 B 互斥，且不做并集。

由以上过程利用 *Matlab* 进行求解，得到导弹 M_1 在烟幕云团内部时，有效遮蔽时长 $t_1 = 0.058s$ ，导弹 M_1 在烟幕云团外部时，有效遮蔽时长 $t_2 = 1.333s$ ，故总有效遮蔽时长 $t = t_1 + t_2 = 1.391s$ 。

5.2 问题二模型的建立与求解

问题二在问题一建立的刚体运动模型和有效遮蔽判定模型的基础上，要求在已知空地导弹 M_1 的初始位置 r_{M_1} ，无人机 FY_1 的初始位置 r_{FY_1} 的情况下，无人机 FY_1 投放一枚烟幕干扰弹后，对真目标的最长有效遮蔽时间。在此过程中，并确定 FY_1 的飞行方向 $\hat{v}_{FY_1}$ 、飞行速度 v_{FY_1} 、烟幕干扰弹投放点 $r_{FY_j}(t_{j,m}^{fall})$ 、烟幕干扰弹起爆点 $P_{j,m}(t_{j,m}^{blow})$ 。可将此问题视为单目标优化问题，通过粒子群算法与模拟退火算法相结合，实现全局搜索和局部精修，进而获得最优解。

本题的具体解题思路如下图：

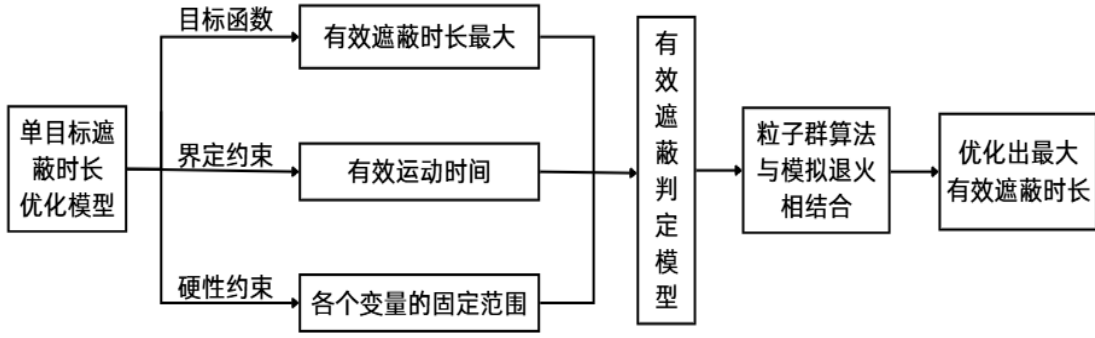


图 4 目标优化模型的流程图

5.2.1 问题二单机单目标优化模型的建立

根据对问题二的分析，问题二的核心为求解最大有效遮蔽时长，其目标方程可表示为：

$$\max_{x \in X} T'(t), \quad T'(t) = \int_{t_{begin}}^{t_{end}} I_i(t; x) dt \quad (24)$$

其中， t_{begin} 表示运动开始时刻， t_{end} 表示运动结束时刻， $T'(t)$ 表示最大有效遮蔽时长， x 表示决策变量，由航向量 θ 、无人机 FY_1 的速度 v_{FY_1} ，烟幕干扰弹投放时间 $t_{j,m}^{fall}$ ，以及烟幕干扰弹引信延时 $t_{j,m}^{delay}$ 构成，即：

$$x = \begin{bmatrix} \theta \\ v_{FY_i} \\ t_{j,m}^{fall} \\ t_{j,m}^{delay} \end{bmatrix} \in X, \quad j=1, m=1 \quad (25)$$

在进行有效遮蔽判定的时间积分中，直接对整个空间直角坐标系的时域进行判定，会增加不必要的计算开销。通过基本的逻辑运算，可知无人机 FY_1 投放的烟幕干扰弹不会超出以下区域球 Ω ：

$$\Omega(O_\Omega, r_\Omega), \quad O_\Omega = \frac{1}{2}(r_{M_0}(0) + O), \quad r_\Omega = \frac{1}{2}\|r_{M_0}(0) - O\| \quad (26)$$

在任意有效运动时刻 t ，烟幕云团的球心到该区域球球心 O_Ω 的距离需满足：

$$\|C_{1,m}(t) - O_\Omega\| \leq r_\Omega \quad (27)$$

烟幕干扰弹在起爆时刻 $t_{1,m}^{blow}$ 要小于导弹命中目标的时刻 $t_{1,m}^{hit}$ ：

$$t_{1,m}^{blow} \leq t_{1,m}^{hit} \quad (28)$$

结合题目中自带硬性约束，可得完整约束条件为：

$$\left\{ \begin{array}{l} 0 \leq \theta < 2\pi \\ 70 \leq v_{FY_1} \leq 140 \\ 0 \leq t_{1,m}^{fall} \leq 20 \\ \frac{1}{2} \leq t_{1,m}^{delay} \leq 20 \\ C_{j,m}(t) \geq z_{floor} + R_{cloud} \\ (M_i(t) - p) \cdot n(p) > 0 \\ \|C_{1,m}(t) - O_{\Omega}\| \leq r_{\Omega} \\ t_{1,m}^{blow} = t_{1,m}^{fall} + t_{1,m}^{delay} \\ t_{1,m}^{blow} \leq t_{1,m}^{hit} \end{array} \right. \quad (29)$$

5.2.2 问题二模型的求解

基于烟幕云团有效遮蔽判定模型的单目标优化模型，可以灵活使用不同的模型和算法进行求解。粒子群优化算法是一种进化计算技术，源于对鸟群捕食的行为研究，其基本思想是通过群体中个体之间的协作和信息共享来寻找最优解^[3]。粒子群优化算法具有较强的全局搜索能力，但在收敛后容易陷入局部最优解，而模拟退火算法在局部区域内可以通过随机扰动跳出局部最优解。故模型二的求解采用粒子群优化算法和模拟退火算法相结合,搜索最大有效遮蔽时长。

具体算法步骤如下图所示：

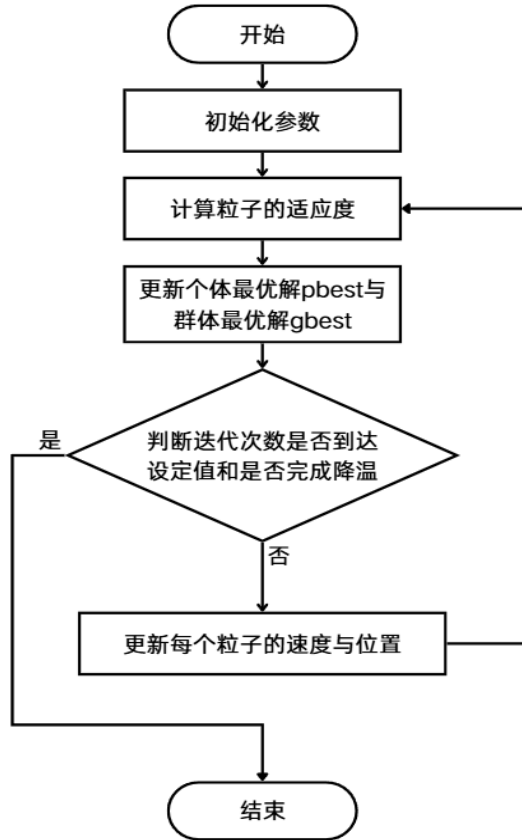


图 5 单目标优化模型算法求解的流程图

step1 初始化粒子群及其参数:

首先, 假定粒子数目为30, 最大迭代次数为150, 权向量 $w = (0.5, 1.5, 1.5)$; 在 $v_1 \in (0, 2)$ 的范围内随机生成粒子群 $x_i = rand(a_n)$ 。

Step2 融合模拟退火算法:

引入初始温度 $T_0 > 0$ 和降温系数 λ , $\lambda \in (0, 1)$, 利用从 T 到 T_λ 实现退火。

Step3 计算粒子适应度:

计算适应函数 $fun(x_i)$ 即每个粒子对应的适应函数值,使各自的历史最优解(个体最优解) p_{besti} 与群体最优解 g_{best} 恢复到初始状态。

Step4 计算个体与全局最优解:

对每个粒子进行个体最优解判定, 先将每个粒子的目标函数值 $fun(x_i)$ 与其经过最好位置的适应值 p_{besti} 比较, 若 $fun(x_i) \geq p_{besti}$, 则将其更新为该粒子的个体最优解, 若此时个体最优解适应值变差, 会以概率 P 变差, 此时概率性实现再次更新, 以此跳出个体局部最优解: 再将所有粒子各自的最高适应度 $\max p_{besti}$ 与全局最高适应度 g_{best} 进行比较, 若 $p_{besti} > g_{best}$, 则将其更新为新的全局最优解。

Step5 判断是否达到迭代次数:

判断迭代次数是否大于 150, 若未超过 150, 粒子群需持续更新自己的速度和位置; 反之, 则输出最终的结果并结束算法。

根据以上算法, 利用 *Matlab* 进行求解, 当有效遮蔽时间最长时, 结果如下表所示:

表 1 问题二求解结果

投放策略	求解结果
无人机的飞行速度 m/s	70.02m/s
无人机的飞行方向(度)	8.43°
烟幕干扰弹投放点 m	(17806.92, 1.03, 1800.00)
烟幕干扰弹起爆点 m	(17872.14, 10.7, 1795.65)
最大有效遮蔽时长 s	4.576

5.3 问题三模型的建立与求解

问题三在问题一建立的刚体运动模型和有效遮蔽判定模型的基础上, 要求在已知空地导弹 M_1 的初始位置 r_{M_1} , 无人机 FY_1 的初始位置 r_{FY_1} 的情况下, 无人机 FY_1 投放 3

枚烟幕干扰弹后，对真目标的最长有效遮蔽时间。

在此过程中，并确定 FY_1 的飞行方向 $\hat{v}_{FY_1}$ 、飞行速度 v_{FY_1} 、烟幕干扰弹投放点 $r_{FY_j}(t_{j,m}^{fall})$ 、烟幕干扰弹起爆点 $P_{j,m}(t_{j,m}^{blow})$ 。可将此问题视为进一步优化已建立的单机单目标优化模型，具体步骤如下：

5.3.1 问题三进一步优化的单机单目标优化模型的建立

题目中要求无人机 FY_1 依次投放 3 枚烟幕干扰弹时，累计对真目标的最长有效遮蔽时间，也就是将单目标优化模型中的目标函数转变为 3 枚烟幕干扰弹的累计有效遮蔽时长，即：

$$T''(t) = \int_{t_{begin}}^{t_{end}} T'_u(t, x') dt = \int_{t_{begin}}^{t_{end}} \min\left(1, \sum_k T'(t, x')\right) dt \quad (30)$$

其中， $T''(t, x')$ 表示真目标在此过程中有效遮蔽的总时长；

$T'_u(t, x')$ 表示在 t 时刻多枚烟幕干扰弹重复遮蔽的情况去重后，真目标与导弹之间的视线是否至少一枚烟幕干扰弹被有效遮蔽。

k 表示第 i 枚烟幕干扰弹 ($i \in \{1, 2, 3\}$)。 x' 表示决策变量，由第 i 枚烟幕干扰弹的投放时间 $t_{1,i}^{fall}$ 和引信延时 $t_{1,i}^{delay}$ 构成即：

$$x = \begin{bmatrix} t_{1,i}^{fall} \\ t_{1,i}^{delay} \end{bmatrix} \in X, \quad i \in \{1, 2, 3\} \quad (31)$$

每架无人机投放两枚烟幕干扰弹的时间至少间隔 1s，故需满足：

$$t_{1,i+1}^{fall} - t_{1,i}^{fall} \geq 1, \quad 1 \leq i \leq 2 \quad (32)$$

3 枚烟幕干扰弹的投放位置和起爆位置未知，当第 i 枚烟幕干扰弹到达引爆时间 $t_{1,i}^{blow}$ 时，第 i 枚烟幕干扰弹的存活时间 t_i 可表示为：

$$t_i \in [t_{1,i}^{blow}, \min(t_{1,i}^{blow} + t_{\max}, t_{1,i}^{blow} + t_{cloud})] \quad , \quad i \in \{1, 2, 3\} \quad (33)$$

结合问题二中的约束条件，可得本模型中的约束条件为：

$$\left\{ \begin{array}{l} 0 \leq \theta < 2\pi \\ 70 \leq v_{FY_1} \leq 140 \\ 0 \leq t_{1,m}^{fall} \leq 20 \\ \frac{1}{2} \leq t_{1,m}^{delay} \leq 20 \\ C_{j,m}(t) \geq z_{floor} + R_{cloud} \\ (M_i(t) - p) \cdot n(p) > 0 \\ t_{1,m}^{blow} = t_{1,m}^{fall} + t_{1,m}^{delay} \\ t_{1,m}^{blow} \leq t_{1,m}^{hit} \\ t_{1,i+1}^{fall} - t_{1,i}^{fall} \geq 1, \quad 1 \leq i \leq 2 \\ t_i \in [t_{1,i}^{blow}, \min(t_{1,i}^{blow} + t_{max}, t_{1,i}^{blow} + t_{cloud})], \quad i \in \{1, 2, 3\} \\ \|C_{1,m}(t) - O_{\Omega}\| \leq r_{\Omega} \end{array} \right. \quad (34)$$

5.3.2 问题三模型的求解

模型三的求解过程与模型二的求解过程相同，均为采用粒子群优化算法和模拟退火算法相结合,遵循单目标优化模型算法求解步骤，求解得出的最长有效遮蔽时间为 6.470s。详细结果如下表和附件`result1.xlsx`。

表 2 问题三求解结果

无人 机运 动方 向	无人 机运 动速 度 (m/s)	干扰 弹编 号	干扰弹投 放点的 x 坐标(m)	干扰弹 投放点 y 坐标 (m)	干扰弹 投放点 z 坐标(m)	干扰弹 起爆点 x 坐标 (m)	干扰弹 起爆点 y 坐标 (m)	干扰弹 起爆点 z 坐标 (m)	有效 干扰 时长 (s)
5.782	132.32	1	17800.03	0.00	1800.00	17800.07	0.01	1800.00	2.560
5.782	132.32	2	17922.92	12.25	1800.00	17922.95	12.45	1800.00	4.560
5.782	132.32	3	19116.36	133.30	1800.00	19226.17	144. 42	1796.59	0.000

5.4 问题四模型的建立与求解

问题四在问题一建立的刚体运动模型和有效遮蔽判定模型的基础上，要求在已知空地导弹 M_1 的初始位置 r_{M_1} ，无人机 $FY_j, (j \in \{1, 2, 3\})$ 的初始位置 r_{FY_j} 的情况下，无人机 FY_j 分别投放 1 烟幕干扰弹后的最长有效遮蔽时间。

在此过程中，并确定三架无人机 FY_j 的飞行方向 $\hat{v}_{FY_i}$ 、飞行速度 v_{FY_i} 、烟幕干扰弹投放点 $r_{FY_j}(t_{j,m}^{fall})$ 、烟幕干扰弹起爆点 $P_{j,m}(t_{j,m}^{blow})$ 。可将此问题视为进一步优化已建立的单机单目标优化模型，具体步骤如下：

5.4.1 问题四多机单目标优化模型的建立

无人机 $FY_j, j \in \{1, 2, 3\}$ 各自依次投放 1 枚烟幕干扰弹时, 累计对真目标的最长有效遮蔽时间为:

$$\begin{aligned} T'''(t) &= T'_u - \lambda(T_{sun}(t) - T'_u), \\ T_{sun}(t) &= \int_{t_{begin}}^{t_{end}} \sum_m^M T'_m(t, x_t) dt, \quad \lambda \in [0.001, 0.005] \end{aligned} \quad (35)$$

其中, $T'_m(t, x) \in \{0, 1\}$ 表示第 m 枚烟幕干扰弹在 t 时刻是否处于有效遮蔽状态; T'_u 表示在目标被命中前被有效遮蔽的总持续时长; $(T_{sun}(t) - T'_u)$ 表示两枚烟幕干扰弹在有效遮蔽持续时长的重叠量; λ 表示每一秒重叠与之等价的有效遮盖秒数的比例。

与此同时, 时间范围需保证在真实可能有效的最小必要区间内进行积分:

$$\begin{aligned} t'_{\min} &= \min_g t_{j,m}^{blow}, \\ t_{\max} &= \min \left(t_{hit}, \max_g (t_{j,m}^{blow}, t_{life}) \right) \end{aligned} \quad (36)$$

其中, t_{life} 表示烟幕干扰弹在起爆后, 烟幕云团被计入有效遮盖状态的时间长度。

x_t 为决策变量, 表示无人机 $FY_j, j \in \{1, 2, 3\}$ 各自依次投放 1 枚烟幕干扰弹时, 烟幕干扰弹的投放时间 $t_{j,1}^{fall}$ 和引信延时 $t_{j,1}^{delay}$, 以及运动过程中无人机 $FY_j, j \in \{2, 3\}$ 的运行速度 v_{FY_j} 和运行方向 θ_j , 即:

$$x_t = \begin{bmatrix} \theta_j \\ v_{FY_j} \\ t_{j,1}^{fall} \\ t_{j,1}^{delay} \end{bmatrix} \in X, \quad j \in \{1, 2, 3\} \quad (37)$$

由于无人机 $FY_j, j \in \{1, 2, 3\}$ 各自依次投放 1 枚烟幕干扰弹的时间顺序未知, 故在时间轴上按照时间顺序给所有相邻的起爆时间 $t_{j,1}^{blow}$ 进行排序, 使相邻的两次起爆时间的最小错峰间隔最少间隔 ϱ 即:

$$t_{1,k+1}^{blow} - t_{1,k}^{blow} \geq \varrho, \quad k \in \{1, 2\} \quad (38)$$

结合问题三, 综合可得完整约束条件为:

$$\left\{ \begin{array}{l} 0 \leq \theta < 2\pi \\ 70 \leq v_{FY_1} \leq 140 \\ 0 \leq t_{1,m}^{fall} \leq 20 \\ \frac{1}{2} \leq t_{1,m}^{delay} \leq 20 \\ C_{j,m}(t) \geq z_{floor} + R_{cloud} \\ (M_i(t) - p) \cdot n(p) > 0 \\ t_{1,m}^{blow} = t_{1,m}^{fall} + t_{1,m}^{delay} \\ t_{1,m}^{blow} \leq t_{1,m}^{hit} \\ t_{1,i+1}^{fall} - t_{1,i}^{fall} \geq 1, \quad 1 \leq i \leq 2 \\ t_i \in [t_{1,i}^{blow}, \min(t_{1,i}^{blow} + t_{max}, t_{1,i}^{blow} + t_{cloud})], \quad i \in \{1, 2, 3\} \\ \|C_{1,m}(t) - O_{\Omega}\| \leq r_{\Omega} \\ t_{1,m+1}^{blow} - t_{1,m}^{blow} \geq \varrho, \quad m \in \{1, 2\} \end{array} \right. \quad (39)$$

5.4.2 问题四模型的求解

模型四的求解过程与模型二的求解过程相同，均为采用粒子群优化算法和模拟退火算法相结合，遵循单目标优化模型算法求解步骤，求解得出的最长有效遮蔽时间为 11.430s。完整策略如下表及附件 *result2.xlsx*。

表 3 问题四求解结果表

无人机运动方向	无人机运动速度(m/s)	干扰弹编号	干扰弹投放点的 x 坐标(m)	干扰弹投放点 y 坐标(m)	干扰弹投放点 z 坐标(m)	干扰弹起爆点 x 坐标(m)	干扰弹起爆点 y 坐标(m)	干扰弹起爆点 z 坐标(m)	有效干扰时长(s)
FY1	7.64 °	95.22	17862.64	8.41	1800.00	17862.64	8.41	1800.00	4.300
FY2	249.89 °	128.02	11770.42	773.04	1400.00	11490.33	8.12	1201.61	2.650
FY3	105.04 °	103.26	5325.24	-488.02	700.00	5169.18	92.97	533.69	4.280

5.5 问题五模型的建立与求解

问题五需要调度五架无人机对三枚来袭导弹完成最优的遮蔽策略规划，通过构建高维决策体系（决策体系包含每架无人机的飞行方向、飞行速度、干扰弹投放时序与起爆时间），建立以三枚导弹总有效遮蔽时长为优化目标的模型，且需满足单机匀速飞行且等高度、烟幕干扰弹弹间投放时间间隔大于等于 1 秒及遮蔽时空有效性等约束条件。

5.5.1 问题五高维多目标优化模型^[4]的建立

根据对问题四的求解可得决策变量 $x\kappa$ ，表示无人机 $FY_j, j \in \{1, 2, 3, 4, 5\}$ ，投放第 i 枚烟幕干扰弹的投放时间 $t_{j,i}^{fall}$ 和引信延时 $t_{j,i}^{delay}$ ，以及无人机 FY_j 运动速度 v_{FY_j} 和航向角 θ_j ，即：

$$x\kappa = \begin{bmatrix} \theta_j \\ v_{FY_j} \\ t_{j,i}^{fall} \\ t_{j,i}^{delay} \end{bmatrix} \quad (40)$$

假设此时的目标函数为 $F(x\kappa)$ ：

$$F(x\kappa) = \begin{bmatrix} F_1 \\ F_2 \\ F_3 \end{bmatrix} \quad (41)$$

- $F_1(x\kappa)$ 表示使有效遮蔽时间最短的一枚烟幕干扰弹最大化，以此保障各个方向都受到保护，即：

$$F_1(x\kappa) = T_{j,m} = \min T_{j,m} \quad (42)$$

- $F_2(x\kappa)$ 表示总体有效遮蔽时长达到最大，以此保障能得到烟幕干扰弹的最优策略：

$$F_2(x\kappa) = \sum T_{j,m} \quad (43)$$

其中， $T_{j,m}$ 表示第 j 架无人机释放的第 m 枚烟幕干扰弹被遮蔽的有效总时长，此时满足的是并集覆盖。

- $F_3(x\kappa)$ 表示总体有效遮蔽时长达到最大，与 $F_2(x\kappa)$ 不同的是， $F_3(x\kappa)$ 满足的是交集覆盖，即：

$$F_3(x\kappa) = \sum_n^{N=1} S_n = \sum_n \sum_{j,m} T_{j,m}^{(n)} \quad (44)$$

其中， S_n 表示将全部烟幕云团对第 n 枚烟幕干扰弹的有效遮蔽时间累加；

$T_{j,m}^{(n)}$ 表示第 j 架无人机释放的第 m 枚烟幕干扰弹独自产生的有效遮蔽时长。

综合可知约束条件为：

$$\left\{ \begin{array}{l} 0 \leq \theta < 2\pi \\ 70 \leq v_{FY_1} \leq 140 \\ 0 \leq t_{j,m}^{fall} \leq 20 \\ \frac{1}{2} \leq t_{j,m}^{delay} \leq 20 \\ C_{j,m}(t) \geq z_{floor} + R_{cloud} \\ (M_i(t) - p) \cdot n(p) > 0 \\ t_{1,m}^{blow} = t_{1,m}^{fall} + t_{1,m}^{delay} \\ t_{j,m}^{blow} \leq t_{j,m}^{hit} \\ t_{j,i+1}^{fall} - t_{j,i}^{fall} \geq 1, \quad 1 \leq i \leq 2 \\ t_i \in [t_{j,i}^{blow}, \min(t_{j,i}^{blow} + t_{max}, t_{j,i}^{blow} + t_{cloud})], \quad i \in \{1, 2, 3\} \\ \|C_{1,m}(t) - O_\Omega\| \leq r_\Omega \\ t_{j,m1+1}^{blow} - t_{j,m1}^{blow} \geq \varrho, \quad m1 \in \{1, 2\} \end{array} \right. \quad (45)$$

其中, $j \in \{1, 2, 3, 4, 5\}, m \in \{1, 2, 3\}$

5.5.2 问题五模型的求解

通过上述“高维多目标优化模型”建立, 求解采用粒子群优化算法和模拟退火算法相结合, 搜索最大有效遮蔽时长。与问题二中的算法流程不同的是, 本题模型的求解通过模拟退火算法中的随机权重 *Tchebycheff* 标准化, 对每步变量施加高斯扰动, 进而能够在一定程度上陷入局部最优解的循环中, 具体算法流程如下所示:

step1 初始化粒子群及其参数:

首先, 假定粒子数目为80, 最大迭代次数为260, 权向量 $w = (1.25, 2.05, 2.05)$; 在 $v_1 \in (0, 2)$ 的范围内随机生成粒子群 $x_i = rand(a_n)$ 。

Step2 融合模拟退火算法:

引入初始温度 $T_0 > 0$ 和降温系数 λ , $\lambda \in (0, 1)$, 利用从 T 到 T_λ 实现退火。

Step3 粒子适应度判定:

评估并计算适应函数 $fun(x_i)$ 即每个粒子对应的适应函数值, 使各自的历史最优解 (个体最优解) p_{besti} 与群体最优解 g_{best} 恢复到初始状态。

Step4 计算个体与全局最优解:

对每个粒子进行个体最优解判定, 先将每个粒子的目标函数值 $fun(x_i)$ 与其经过最好位置的适应值 p_{besti} 比较, 若 $fun(x_i) \geq p_{besti}$, 则将其更新为该粒子的个体最优解, 若此时 $fun(x_i) \gg p_{besti}$, 则将所有粒子与已经存在的存档合并, 重新进行标准化。

Step5 高斯扰动:

对于本题的多目标优化模型, 可首先应用模拟退火算法中的随机权重

*Tchebycheff*标准化, 此时退火温度是大 $T_0 > 0$ 开始, 对每步变量施加高斯扰动。实现再次更新, 以此重复个体局部最优解和全局最优解的更新。

Step6 判断是否达到迭代次数:

判断迭代次数是否大于260, 若未超过260, 粒子群持续更新其速度和位置; 反之, 则输出最终的结果并结束算法。

根据以上算法, 利用 *Matlab* 进行求解, 计算得出最大有效遮蔽时间为22.850s, 其中无人机编号、运动方向以及运动速度如下图所示, 完整详细结果见附件 *result3.xlsx*。

表 4 问题五部分求解结果

无人机编号	无人机运动方向	无人机运动速度 (m/s)	烟幕干扰弹编号
FY1	3.681°	125.88	1
FY1	3.681°	125.88	2
FY1	3.681°	125.88	3
FY2	238.879°	92.51	1
FY2	238.879°	92.51	2
FY2	238.879°	92.51	3
FY3	78.171°	101.292	1
FY3	78.171°	101.292	2
FY3	78.171°	101.292	3
FY4	293.289°	79.305	1
FY4	293.289°	79.305	2
FY4	293.289°	79.305	3
FY5	96.574°	132.959	1
FY5	96.574°	132.959	2
FY5	96.574°	132.959	3

六、模型的分析与检验

针对问题一,问题一中建立的“有效遮蔽判定模型”——它涉及烟幕云团对导弹之间的有效遮蔽时长 $T_{j,m}$ 对真目标圆柱体离散精度的敏感性,即对于圆柱体可见外轮廓取样点的个数影响着最终求解出的有效遮蔽时长 $T_{j,m}$ 。

当从圆柱体可见外轮廓的取样点从粗颗粒度向细颗粒度聚拢时,相应的有效遮蔽时长 $T_{j,m}$ 的趋势变化如下图所示:

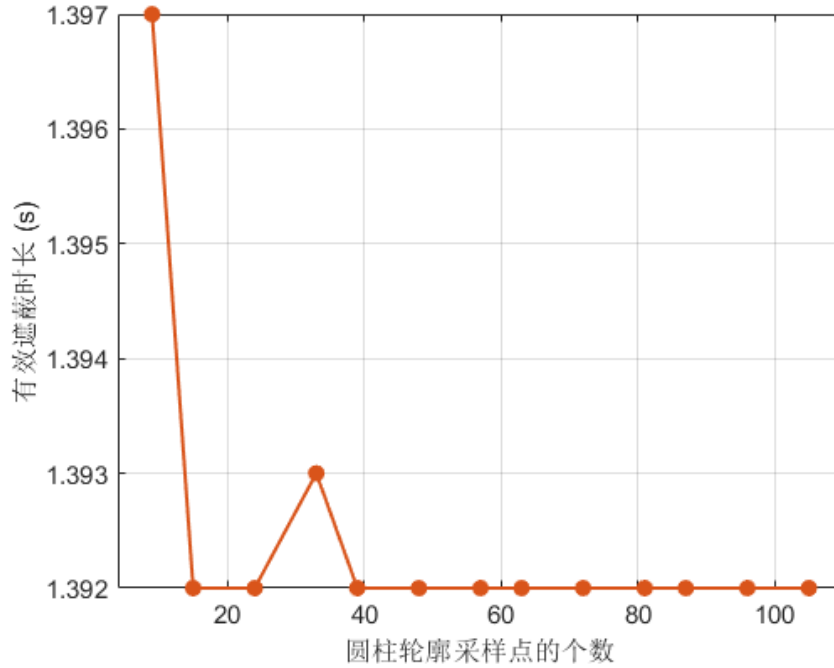


图 6 取样点不同的情况下有效遮蔽时长趋势图

通过观察图像可得:

当起始取样点在 8 到 16 点之间时,有效遮蔽时长从约为 1.397s 迅速下降到 1.392 秒;

当取样点增加至 16 到 20 点之间时,有效遮蔽时长接近稳定;

当取样点为 32 点时,有效遮蔽时长存在细小数值波动;

之后在取样点大于等于 40 点时,有效遮蔽时长基本贴近一条水平线。

这些趋势表面,说明在粗采样时,高估了有效遮蔽时长;在极少取样点的附近存在细小数值波动,属于数值上的离散噪声,不改变整体单调收敛趋势;当取样点大于等于 40 点时,有效遮蔽时长基本贴近一条水平线,满足了 $|\Delta T|$ 很小且连续满足的不敏感判据。

灵敏度检验的结果:

从取样点对有效遮蔽时长的灵敏度可以从结果看出来,取样点的离散精度一旦达到阈值(40),有效遮蔽时长对取样点数几乎不敏感。

七、模型的评价、改进与推广

7.1 模型优点

1.刚体运动模型将导弹、无人机和烟幕当成具体的点。采用这种方法计算更快、相关的公式好写、不容易出现错误；对于第五问的多架协同也便于相加，结果更加通俗易懂。

2.模拟退火算法与粒子群算法相结合可以更好的进行模型的求解，模拟退火算法也可以防止局部陷入最优解，两者互补，最终的收敛更快，实现起来更加简单可行。

3.多目标优化模型能把目标量化交给 Matlab 权衡取舍，能够迅速的在复杂的决策变量与约束条件给出可行亦或是更优的方案，并且可以实现多个变量的重复计算。

7.2 模型缺点

1.目标优化模型需确立两套参数，计算量更大，若选取不当会过早收敛或这过度随机，影响最终结果的稳定性。

2.模拟退火算法与粒子群算法相结合对权重和参数较敏感，容易过早收敛到局部最优，对模型误差与噪声描述不够精细。

7.3 模型改进

1. 采用自定义参数的模拟退火算法与粒子群算法相结合混合搜索，先粗搜后精修；引入鲁棒多目标优化与近似模型相结合，实现理想环境下的单目标求解”进阶为“实战环境下的多目标鲁棒决策”。

2. 在目标优化模型中加入空气阻力影响时，需要将自由落体运动假设进阶为加入空气阻力的微分方程的运动模型，采用特殊方法求解干扰弹轨迹与云团下沉的路径，通过蒙特卡洛采样法评估干扰弹策略的鲁棒性，最终生成既可以匹配实际的外部环境又具备抵抗外界干扰能力的烟幕投放方案。

八、参考文献

[1] 梅雨. 基于模拟退火粒子群算法的微电网优化调度[J]. 现代信息科技, 2025, 9(05): 134-138. DOI: 10.19850/j.cnki.2096-4706.2025.05.025.

[2] 司呈勇, 杨东升, 田红军, 等. 一种改进的模拟退火粒子群算法在有约束函数优化问题中的应用[C]//中国自动化学会智能自动化专业委员会. 2011 年中国智能自动化学术会议论文集（第一分册）. 同济大学 电子与信息工程学院; 同济大学 嵌入式系统与服务计算教育部重点实验室; 上海经济和信息化委员会; 2011: 182-186.

[3] 周驰, 高海兵, 高亮等. 粒子群优化算法[J]. 计算机应用研究, 2003(12): 7-11.. 周驰; 高海兵; 高亮; 章万国.

[4] 杨帅宇. 无人机路径优化问题的多目标规划模型与算法研究[D]. 长春工业大学, 2025. DOI: 10.27805/d.cnki.gccgy.2025.000729.

- 本参赛队未使用任何 AI 工具。

附录

附录 1

介绍：支撑材料的文件列表

文件名	含义
rsult1.xlsx	问题三求解结果
rsult2.xlsx	问题四求解结果
rsult3.xlsx	问题五求解结果
question1.m	问题一完整代码
question2.m	问题二完整代码
question3.m	问题三完整代码
question4.m	问题四完整代码
question5.m	问题五完整代码
lingmingdu.m	灵敏度分析完整代码

附录 2

利用：matlab 求解第一问代码

```
function question1()
% 有效时间 = A + B（不做并集，不填缝）
% A: 导弹在云团内部
% B: 导弹在云团外部 且 满足“角度覆盖 + 线段与球体相交”（前向基准 =
walkway_msl）
% A 与 B 互斥（实现上 B 只在 ~inside 时计），因此总有效 = total_A +
total_B
clc;
%% （标量/常量）
grape_g = 9.8; % m/s^2
mango_uav = 120; % UAV 水平速度
banana_msl = 300; % 导弹速度
apple_cloud = 10; % 云团半径 (m)
peach_sink = 3; % 云团下沉速度 (m/s)
kiwi_drop = 1.5; % 投放时刻 (s)
lemon_fuse = 3.6; % 引信时延 (s)
lychee_boom = kiwi_drop + lemon_fuse; % 5.1 s
papaya_end = lychee_boom + 20.0; % 25.1 s

orange_nphi = 720; % 圆周采样
cherry_nz = 11; % 高度采样
fig_dt = 0.001; % 时间步长
berry_eps = 1e-12; % 数值余量

melon_r_cyl = 7; % 圆柱半径
melon_h_cyl = 10; % 圆柱高度
```

```

%% （几何/向量/点）
plaza_origin = [0,0,0];
hangar_uav0 = [17800,0,1800]; % FY1 初始
silo_msl0 = [20000,0,2000]; % M1 初始

% UAV 航向（水平指向原点）
corridor_uav_xy = (plaza_origin(1:2) - hangar_uav0(1:2));
corridor_uav_xy = corridor_uav_xy / norm(corridor_uav_xy);
corridor_uav = [corridor_uav_xy, 0];

% 导弹方向（恒指原点；列向量）
walkway_msl = (plaza_origin - silo_msl0);
walkway_msl = walkway_msl / norm(walkway_msl);
walkway_msl = walkway_msl(:);

%% 关键位置
rooftop_drop = hangar_uav0 + mango_uav * kiwi_drop * corridor_uav;
chimney_c0 = rooftop_drop + mango_uav * lemon_fuse * corridor_uav
+ [0,0,-0.5*grape_g*lemon_fuse^2];
stair_m51 = silo_msl0 + banana_msl * lychee_boom * walkway_msl.';

fprintf('--- 关键时空 ---\n');
fprintf('P_drop (t=1.5s) = (%.3f, %.3f, %.3f)\n', rooftop_drop);
fprintf('C0 (t=5.1s) = (%.3f, %.3f, %.3f)\n', chimney_c0);
fprintf('M (t=5.1s) = (%.3f, %.3f, %.3f)\n\n', stair_m51);

%% 圆柱轮廓采样（前向可见可选）
plaza_cyl_base = [0, 200, 0]; % 真目标底面圆心
balcony_zlvl = linspace(0, melon_h_cyl, cherry_nz);
arch_phi = linspace(0, 2*pi, orange_nphi+1); arch_phi(end) = [];
arch_cp = cos(arch_phi);
arch_sp = sin(arch_phi);

skyline_Pts = zeros(3, orange_nphi*cherry_nz);
col = 0;
for iz = 1:cherry_nz
    z = balcony_zlvl(iz);
    x = plaza_cyl_base(1) + melon_r_cyl * arch_cp;
    y = plaza_cyl_base(2) + melon_r_cyl * arch_sp;
    zvec = plaza_cyl_base(3) + z * ones(1, orange_nphi);
    idx = col + (1:orange_nphi);
    skyline_Pts(:,idx) = [x; y; zvec];
    col = col + orange_nphi;

```



```

end
gate_front_only = true;

%% 遍历时间 [5.1, 25.1]
terrace_ts = lychee_boom:fig_dt:papaya_end;
gate_A = false(size(terrace_ts)); % A: 导弹在云内
gate_B = false(size(terrace_ts)); % B: 导弹在云外且几何口径有效

chimney_c0_col = chimney_c0(:);

for k = 1:numel(terrace_ts)
t = terrace_ts(k);

% 位置
atrium_M = (silo_msl0 + banana_msl * t * walkway_msl.'.'); % 3x1
atrium_C = chimney_c0_col + [0;0;-peach_sink*(t - lychee_boom)]; %
3x1

% A: 是否在云内（无条件计时）
hall_dMC = norm(atrium_C - atrium_M);
inside_now = (hall_dMC <= apple_cloud + 1e-12);
gate_A(k) = inside_now;

% B: 仅在“云外”情况下进行判定
if ~inside_now
% 云团半角与方向
hallway_MC = atrium_C - atrium_M;
tunnel_d = sqrt(sum(hallway_MC.^2));
dome_alpha = asin(min(1, apple_cloud / tunnel_d));
elevator_vC = hallway_MC / tunnel_d; % 单位向量 (M->C)

% 导弹->轮廓向量
facade_V = bsxfun(@minus, skyline_Pts, atrium_M);
if gate_front_only
walkway_rep = repmat(walkway_msl, 1, size(facade_V,2));
front = sum(facade_V .* walkway_rep, 1) > 0;
if ~any(front)
gate_B(k) = false;
continue;
end
facade_V = facade_V(:, front);
end

% 角度覆盖

```

```

        facade_Vn = sqrt(sum(facade_V.^2, 1)); facade_Vn = max(facade_Vn,
eps);
        facade_Vu = bsxfun(@rdivide, facade_V, facade_Vn);
        lobby_VC = repmat(elevator_vC, 1, size(facade_Vu,2));
        brick_dots = sum(facade_Vu .* lobby_VC, 1);
        brick_dots(brick_dots>1)=1; brick_dots(brick_dots<-1)=-1;
        beam_betas = acos(brick_dots);
        if max(beam_betas) > dome_alpha + berry_eps
            gate_B(k) = false;
            continue;
        end

        % 包含性: 线段-球体相交 (此处 tunnel_d > apple_cloud)
        atrium_ustar = tunnel_d * brick_dots;
        alley_dperp = tunnel_d * sqrt(max(0, 1 - brick_dots.^2));
        courtyard_inside = (alley_dperp <= apple_cloud + 1e-12);
        column_delta = sqrt(max(0, apple_cloud^2 - alley_dperp.^2));
        lintel_ul = atrium_ustar - column_delta;
        lintel_ur = atrium_ustar + column_delta;
        bridge_L = facade_Vn;
        firewall_inter = courtyard_inside & (lintel_ur >= -1e-12) &
(lintel_ul <= bridge_L + 1e-12);
        gate_B(k) = all(firewall_inter);
        else
            gate_B(k) = false; % 在云内不计 B
        end
    end

    % 分段与统计
    [blocks_A, total_A] = stream_segments(terrace_ts, gate_A);
    [blocks_B, total_B] = stream_segments(terrace_ts, gate_B);
    total_sum = total_A + total_B; % 只做求和, 不做并集

    % 打印
    fprintf('[A] 导弹在云团内部: \n'); wave_report(blocks_A, total_A);
    fprintf('[B] 导弹在云外且“角度覆盖+线段相交”成立: \n');
    wave_report(blocks_B, total_B);
    fprintf('—— 总有效 (A + B, 不含间隙、不做并集) = %.3f s ——\n',
total_sum);
    fprintf('t = 5.1 s 是否处于 A: %s, 处于 B: %s\n',
water_pick(gate_A(1), '是', '否'), water_pick(gate_B(1), '是', '否'));
end

```

附录 3

利用：matlab 求解第二问代码

```
function question2()
% 第二问 (PSO 优化)
% 通过‘角度覆盖 + 线段-球相交’的严格口径判定有效遮蔽,
% 用 PSO 优化 x = [theta_deg, v, t_drop, t_fuse] 以最大化总有效遮蔽
时长。

clc;
%% 常量
school_g = 9.8; % m/s^2
library_Rcloud = 10; % 云团半径 m
bus_sink = 3; % 云团下沉 m/s
train_msl = 300; % 导弹速度 m/s

campus_0 = [0,0,0];
dorm_F0 = [17800,0,1800]; % FY1 初始
gate_M0 = [20000,0,2000]; % M1 初始
school_r = 7; school_h = 10; campus_base = [0,200,0];

% 导弹方向 (恒定直指原点; 列向量)
road_um = (campus_0 - gate_M0); road_um = road_um / norm(road_um);
road_um = road_um(:);

%% 采样参数 (粗 / 细)
tram_coarse.n_phi = 360; tram_coarse.n_z = 7; tram_coarse.dt =
0.009; % PSO 评估 (快)
tram_fine.n_phi = 720; tram_fine.n_z = 11; tram_fine.dt = 0.001; %
复核 (严)
road_front = true; % 只判前向可见轮廓 (基准 = road_um)

Pts_tram_coarse = build_cylinder_points(campus_base, school_r,
school_h, tram_coarse.n_phi, tram_coarse.n_z);
Pts_tram_fine = build_cylinder_points(campus_base, school_r,
school_h, tram_fine.n_phi, tram_fine.n_z);

%%决策变量范围
lane_lb = [ 0, 70, 0.0, 0.5];
lane_ub = [ 360, 140, 20.0, 8.0];
lane_rng = lane_ub - lane_lb;

%% 粒子群参数
rng(42);
```

```

china_desk = 80; % 粒子数
america_table = 80; % 迭代数

france_chair = 2.05;
germany_chair = 2.05;
italy_desk = france_chair + germany_chair;
japan_table = 2/(italy_desk - 2 + sqrt(italy_desk^2 -
4*italy_desk));

brazil_desk = 0.30 * lane_rng;
brazil_table = -brazil_desk;
india_desk = 0.06 * lane_rng;
india_table = -india_desk;
canada_table = 1.5;

spain_desk = 15;
russia_table = [2, 0.8, 0.15, 0.08];

egypt_desk = zeros(china_desk,4);
mexico_table = zeros(china_desk,4);
turkey_desk = zeros(china_desk,1);

for i=1:china_desk
    egypt_desk(i,:) = lane_lb + rand(1,4).*(lane_ub-lane_lb);
    egypt_desk(i,1) = mod(egypt_desk(i,1), 360);
    mexico_table(i,:) = zeros(1,4);
end
egypt_desk(1,:) = [180, 120, 1.5, 3.6];

sweden_table = egypt_desk;
norway_desk = -inf(china_desk,1);
finland_table = [];
denmark_desk = -inf;

for i=1:china_desk
    f = fitness_time(egypt_desk(i,:),
gate_M0,road_um,dorm_F0,school_g, library_Rcloud,bus_sink, ...
    campus_base,school_r,school_h, Pts_tram_coarse, road_front,
tram_coarse.dt);
    norway_desk(i) = f;
end
[denmark_desk, idx] = max(norway_desk);
finland_table = sweden_table(idx,:);
fprintf('Iter %2d best = %.3f s\n', 0, denmark_desk);

```

```

%% 主循环
for it=1:america_table
    tau = 1 - (it / america_table);
    wV = tau^canada_table;
    Vmax = india_desk + wV .* (brazil_desk - india_desk);
    Vmin = -Vmax;

    for i=1:china_desk
        if turkey_desk(i) >= spain_desk
            jitter = [russia_table(1)*randn, russia_table(2)*randn, ...
                russia_table(3)*randn, russia_table(4)*randn];
            egypt_desk(i,:) = egypt_desk(i,:) + jitter;
            egypt_desk(i,:) = max(min(egypt_desk(i,:), lane_ub), lane_lb);
            egypt_desk(i,1) = mod(egypt_desk(i,1), 360);
            mexico_table(i,:) = 0;
            turkey_desk(i) = 0;
        end

        r1 = rand(1,4); r2 = rand(1,4);
        mexico_table(i,:) = japan_table * ( mexico_table(i,:) ...
            + france_chair*r1.*(sweden_table(i,:) - egypt_desk(i,:)) ...
            + germany_chair*r2.*(finland_table - egypt_desk(i,:)) );

        mexico_table(i,:) = max(mexico_table(i,:), Vmin);
        mexico_table(i,:) = min(mexico_table(i,:), Vmax);

        egypt_desk(i,:) = egypt_desk(i,:) + mexico_table(i,:);
        over = egypt_desk(i,:) > lane_ub; under = egypt_desk(i,:) <
        lane_lb;
        egypt_desk(i,over) = lane_ub(over); mexico_table(i,over) = -
        0.5*mexico_table(i,over);
        egypt_desk(i,under) = lane_lb(under); mexico_table(i,under) = -
        0.5*mexico_table(i,under);
        egypt_desk(i,1) = mod(egypt_desk(i,1), 360);

        f = fitness_time(egypt_desk(i,:),
            gate_M0,road_um,dorm_F0,school_g, library_Rcloud,bus_sink, ...
            campus_base,school_r,school_h, Pts_tram_coarse, road_front,
            tram_coarse.dt);

        if f > norway_desk(i) + 1e-12
            norway_desk(i) = f; sweden_table(i,:) = egypt_desk(i,:);
            turkey_desk(i) = 0;
        end
    end
end

```

```

else
turkey_desk(i) = turkey_desk(i) + 1;
end

if f > denmark_desk
denmark_desk = f; finland_table = egypt_desk(i,:);
end
end
fprintf('Iter %2d best = %.3f s\n', it, denmark_desk);
end

best_x = finland_table; best_fit_coarse = denmark_desk;

%% 最优解上“细判定”复核并打印
theta_deg = best_x(1); v = best_x(2); t_drop = best_x(3); t_fuse =
best_x(4);
bus_dir = [cosd(theta_deg); sind(theta_deg); 0];
P_drop = dorm_F0(:) + v * t_drop * bus_dir;
C0 = P_drop + v * t_fuse * bus_dir + [0;0;-0.5*school_g*t_fuse^2];
t_expl = t_drop + t_fuse;

[blocks, total_cov] =
cover_intervals(gate_M0,road_um,C0,library_Rcloud,bus_sink, ...
t_expl, t_expl+20, ...
Pts_tram_fine, campus_base,school_r,school_h, ...
road_front, tram_fine.dt);
M_at_expl = gate_M0 + (train_msl * t_expl) * (road_um. ');

fprintf('\n=== PSO 最优解（细判定复核，不含区域球约束） ===\n');
fprintf('theta = %.3f° v = %.3f m/s t_drop = %.3f s t_fuse = %.3f
s t_expl = %.3f s\n', ...
theta_deg, v, t_drop, t_fuse, t_expl);
fprintf('P_drop = (%.3f, %.3f, %.3f)\n', P_drop);
fprintf('C0 = (%.3f, %.3f, %.3f)\n', C0);
fprintf('M(t_expl) = (%.3f, %.3f, %.3f)\n', M_at_expl);
fprintf('粗判定最优时长 = %.3f s\n', best_fit_coarse);

if isempty(blocks)
fprintf('细判定：无有效遮蔽（总时长 = 0 s）。\n');
else
fprintf('细判定：有效遮蔽时间段（[t_expl, t_expl+20]）\n');
sumlen = 0;
for i=1:size(blocks,1)
t0 = blocks(i,1); t1 = blocks(i,2);

```

```

        fprintf(' 区间 %2d: [%.3f s, %.3f s] 时长 = %.3f s\n', i, t0, t1,
t1-t0);
        sumlen = sumlen + (t1 - t0);
    end
    fprintf('总有效遮蔽时长（细判定） = %.3f s\n', sumlen);
end
end

```

附录 4

利用：matlab 求解第三问代码

```

clear; close all; clc;
river_M1_0 = [20000; 0; 2000];
river_FY1_0 = [17800; 0; 1800];
river_v_M1_mag = 300;
water_g = 9.8;

water_R_T = 7;
water_H_T = 10;
water_C_T = [0; 200; 0];

water_R_cloud = 10;

water_constants.river_M1_0 = river_M1_0;
water_constants.river_FY1_0 = river_FY1_0;
water_constants.river_v_M1_mag = river_v_M1_mag;
water_constants.water_g = water_g;
water_constants.water_R_T = water_R_T;
water_constants.water_H_T = water_H_T;
water_constants.water_C_T = water_C_T;
water_constants.water_R_cloud = water_R_cloud;

%% === 决策变量（共 8 个）===
% x(1): theta_FY1 - 无人机飞行方向（弧度）
% x(2): v_FY1 - 无人机飞行速度（米/秒）
% x(3): t_fly_1 - 第一次投放前的飞行时间
% x(4): t_fuse_1 - 第一个烟幕弹的引信时间
% x(5): t_fly_2 - 第二次投放前的飞行时间
% x(6): t_fuse_2 - 第二个烟幕弹的引信时间
% x(7): t_fly_3 - 第三次投放前的飞行时间
% x(8): t_fuse_3 - 第三个烟幕弹的引信时间
river_nvars = 8;

```

```

    river_theta_min = 0; river_theta_max = 0.2; % 方向角范围（可按需调整）
    river_v_min = 70; river_v_max = 140; % 速度范围
    river_tfly_min = 0; river_tfly_max = 10; % 投放前飞行时间
    river_tfuse_min = 0; river_tfuse_max = 10; % 引信时间

    river_lb = [river_theta_min, river_v_min, river_tfly_min,
    river_tfuse_min, river_tfly_min, river_tfuse_min, river_tfly_min,
    river_tfuse_min];
    river_ub = [river_theta_max, river_v_max, river_tfly_max,
    river_tfuse_max, river_tfly_max, river_tfuse_max, river_tfly_max,
    river_tfuse_max];

    % 相邻两次投放至少间隔 1 s:  $t_{fly\_2} \geq t_{fly\_1} + 1$ ;  $t_{fly\_3} \geq t_{fly\_2} + 1$ 
    river_A = [0 0 1 0 -1 0 0 0; %  $t_{fly\_1} - t_{fly\_2} \leq -1$ 
    0 0 0 0 1 0 -1 0]; %  $t_{fly\_2} - t_{fly\_3} \leq -1$ 
    river_b = [-1; -1];

    %% === 适应度函数 ===
    window_fitfun = @(x) window_fitness_Q3(x, water_constants);
    window_options = optimoptions('ga', ...
    'PopulationSize', 100, ...
    'MaxGenerations', 50, ...
    'FunctionTolerance', 1e-4, ...
    'ConstraintTolerance', 1e-4, ...
    'Display', 'iter', ...
    'CrossoverFraction', 0.9, ...
    'MutationFcn', {@mutationuniform, 0.1}, ...
    'PlotFcn', [], ...
    'UseParallel', true);
    [window_x_optimal, window_fval] = ga(window_fitfun, river_nvars,
    river_A, river_b, [], [], river_lb, river_ub, [], window_options);

    %% 结果解析
    river_theta_opt = window_x_optimal(1);
    river_v_opt = window_x_optimal(2);
    river_tfly_all_opt = [window_x_optimal(3), window_x_optimal(5),
    window_x_optimal(7)]; % 第1,2,3枚
    river_tfuse_all_opt = [window_x_optimal(4), window_x_optimal(6),
    window_x_optimal(8)];

    river_v_vec = river_v_opt * [cos(river_theta_opt);
    sin(river_theta_opt); 0];

```



```

water_P_drops = zeros(3, 3);
water_P_dets = zeros(3, 3);
for k = 1:3
    river_tfly = river_tfly_all_opt(k);
    river_tfuse = river_tfuse_all_opt(k);
    water_P_drops(:, k) = river_FY1_0 + river_v_vec * river_tfly;
    water_P_dets(:, k) = water_P_drops(:, k) + river_v_vec *
river_tfuse + [0; 0; -0.5 * water_g] * river_tfuse^2;
end

max_shielding_time_total = -window_fval;

[~, river_individual_times_sorted, river_detonation_order] =
window_analyze_shielding_time(window_x_optimal, water_constants);
river_individual_times = zeros(1,3);
river_individual_times(river_detonation_order) =
river_individual_times_sorted;

fprintf('飞行方向: %.3f 度\n', rad2deg(river_theta_opt));
fprintf('飞行速度: %.2f m/s\n', river_v_opt);
for k = 1:3
    fprintf('第 %d 棵烟幕弹\n', k);
    fprintf(' 投放前飞行时间: %.3f s\n', river_tfly_all_opt(k));
    fprintf(' 烟幕弹引信时间: %.3f s\n', river_tfuse_all_opt(k));
    fprintf(' 投放点坐标 (X,Y,Z): (%.2f, %.2f, %.2f) m\n',
water_P_drops(1, k), water_P_drops(2, k), water_P_drops(3, k));
    fprintf(' 起爆点坐标 (X,Y,Z): (%.2f, %.2f, %.2f) m\n',
water_P_dets(1, k), water_P_dets(2, k), water_P_dets(3, k));
    fprintf(' [独立有效时长贡献]: %.3f s\n',
river_individual_times(k));
end
fprintf('\n 找到的最大有效遮蔽时长: %.3f 秒\n',
max_shielding_time_total);

%% 函数: 适应度
function neg_T_effective = window_fitness_Q3(x, constants)
[T_total, ~, ~] = window_analyze_shielding_time(x, constants);
neg_T_effective = -T_total;
end

%% 函数: 评估遮蔽总时长 (严格口径)
function [T_total, T_individual_sorted, det_order] =
window_analyze_shielding_time(x, constants)

```

```

theta_FY1 = x(1); v_FY1 = x(2);
t_fly_all = [x(3), x(5), x(7)];
t_fuse_all = [x(4), x(6), x(8)];

river_M1_0 = constants.river_M1_0;
river_FY1_0 = constants.river_FY1_0;
river_v_M1_mag = constants.river_v_M1_mag;
water_g = constants.water_g;
water_R_T = constants.water_R_T;
water_H_T = constants.water_H_T;
water_C_T = constants.water_C_T;
water_R_cloud = constants.water_R_cloud;

u_M1 = ([0;0;0] - river_M1_0) / norm([0;0;0] - river_M1_0);
v_M1 = river_v_M1_mag * u_M1;
P_M1_t = @(t) river_M1_0 + v_M1 * t;

v_FY1_vec = v_FY1 * [cos(theta_FY1); sin(theta_FY1); 0];
P_FY1_t = @(t) river_FY1_0 + v_FY1_vec * t;

t_det_unsorted = t_fly_all + t_fuse_all;
[t_det_sorted, det_order] = sort(t_det_unsorted);
t_fly_sorted = t_fly_all(det_order);

P_bomb_t = @(t, t_fly) P_FY1_t(t) + [0;0;-0.5*water_g] .* (t -
t_fly).^2 .* (t >= t_fly);
P_cloud_t = @(t, t_fly, t_det) P_bomb_t(min(t, t_det), t_fly) +
[0;0;-3] .* (t - t_det) .* (t >= t_det);

P_cloud1_t = @(t) P_cloud_t(t, t_fly_sorted(1), t_det_sorted(1));
P_cloud2_t = @(t) P_cloud_t(t, t_fly_sorted(2), t_det_sorted(2));
P_cloud3_t = @(t) P_cloud_t(t, t_fly_sorted(3), t_det_sorted(3));

% 目标关键点
K_T = [ water_C_T(1), water_C_T(1), water_C_T(1)+water_R_T,
water_C_T(1)-water_R_T;
water_C_T(2)-water_R_T, water_C_T(2)+water_R_T, water_C_T(2),
water_C_T(2);
water_C_T(3), water_C_T(3), water_C_T(3), water_C_T(3) ];
K_T = [K_T, K_T + [0;0;water_H_T]];
num_key_points = size(K_T, 2);

% 仿真时间窗
dt_sim = 0.01;

```

```

t_sim_start = t_det_sorted(1);
t_sim_end = min(t_det_sorted(3) + 20, t_det_sorted(1) + 25);

if isempty(t_sim_start) || (t_sim_end <= t_sim_start)
T_total = 0; T_individual_sorted = [0, 0, 0]; return;
end

sim_time_vec = t_sim_start : dt_sim : t_sim_end;

shielded_steps_total = 0;
shielded_steps_individual = [0, 0, 0];

for t = sim_time_vec
P_M1_current = P_M1_t(t);

idx1 = []; idx2 = []; idx3 = [];
if t >= t_det_sorted(1) && t < t_det_sorted(1) + 20
idx1 = window_get_occluded_indices_strict(P_M1_current,
P_cloud1_t(t), K_T, water_R_cloud, u_M1, true);
if length(unique(idx1)) == num_key_points
shielded_steps_individual(1) = shielded_steps_individual(1) + 1;
end
end
if t >= t_det_sorted(2) && t < t_det_sorted(2) + 20
idx2 = window_get_occluded_indices_strict(P_M1_current,
P_cloud2_t(t), K_T, water_R_cloud, u_M1, true);
if length(unique(idx2)) == num_key_points
shielded_steps_individual(2) = shielded_steps_individual(2) + 1;
end
end
if t >= t_det_sorted(3) && t < t_det_sorted(3) + 20
idx3 = window_get_occluded_indices_strict(P_M1_current,
P_cloud3_t(t), K_T, water_R_cloud, u_M1, true);
if length(unique(idx3)) == num_key_points
shielded_steps_individual(3) = shielded_steps_individual(3) + 1;
end
end

if length(union(union(idx1, idx2), idx3)) == num_key_points
shielded_steps_total = shielded_steps_total + 1;
end
end

T_total = shielded_steps_total * dt_sim;

```

```

T_individual_sorted = shielded_steps_individual * dt_sim;
end

%% 函数：严格口径（角度覆盖 + 线段-球相交 + 前向可见）
function occluded_indices =
window_get_occluded_indices_strict(P_M1, P_cloud, K_T, R_cloud, u_m,
front_only)
    % P_M1: 3x1 导弹位置
    % P_cloud: 3x1 云心
    % K_T: 3xN 目标关键点
    % R_cloud: 云半径
    % u_m: 3x1 导弹速度方向单位向量（前向筛选基准）
    % front_only: true/false 是否启用前向可见筛选

    tol = 1e-12;

    V = P_cloud - P_M1; % 导弹→云心
    d = norm(V);

    if d <= R_cloud + tol
        % 导弹在云内：直接判所有关键点被遮蔽
        occluded_indices = 1:size(K_T, 2);
        return;
    end

    vC = V / d;
    alpha = asin(min(1, R_cloud / d));

    occluded_indices = [];
    for k = 1:size(K_T, 2)
        W = K_T(:,k) - P_M1; % 导弹→关键点
        L = norm(W);
        if L < tol, continue; end
        What = W / L;

        % 前向可见
        if front_only && dot(W, u_m) <= 0
            continue;
        end

        % 角度覆盖: beta <= alpha
        cosb = max(-1, min(1, dot(What, vC)));
        beta = acos(cosb);
        if beta > alpha + tol

```

```

        continue;
    end

    % 线段-球体相交：基于最近距离 + 交弦半长
    ustar = d * cosb; % 云心在视线方向上的投影距离
    d_perp = d * sqrt(max(0, 1 - cosb^2)); % 云心到射线的最短距离
    if d_perp > R_cloud + tol
        continue;
    end
    delta = sqrt(max(0, R_cloud^2 - d_perp^2)); % 交弦半长
    u_l = ustar - delta; % 入点参数
    u_r = ustar + delta; % 出点参数

    % 与线段 [0, L] 有交则判为遮蔽
    if (u_r >= -tol) && (u_l <= L + tol)
        occluded_indices = [occluded_indices, k]; %#ok<AGROW>
    end
end
end
end

```

附录 5

利用：matlab 求解第四问代码

```

clc; clear; close all;
clc; clear; close all;

%% 常量/题设
river_P_M1_0 = [20000; 0; 2000];
river_v_M1_mag = 300;

% 新演员，3 架无人机初始信息
river_P_FY1_0 = [17800; 0; 1800];
river_P_FY2_0 = [12000; 1400; 1400];
river_P_FY3_0 = [6000; -3000; 700];

water_g = 9.8;
water_R_T = 7;
water_H_T = 10;
water_C_T = [0; 200; 0];
water_R_cloud = 10;

water_constants.river_P_M1_0 = river_P_M1_0;
water_constants.river_v_M1_mag = river_v_M1_mag;
water_constants.river_P_FY_0s = {river_P_FY1_0, river_P_FY2_0,
river_P_FY3_0}; % 注意：原名带双下划线

```

```

water_constants.water_g = water_g;
water_constants.water_R_T = water_R_T;
water_constants.water_H_T = water_H_T;
water_constants.water_C_T = water_C_T;
water_constants.water_R_cloud = water_R_cloud;

%% 决策变量 (共 12 个)
river_nvars = 12;
river_lb = [ ...
0, 70, 0, 0, ... % FY1
pi+atan(1/10), 70, 0, 0, ... % FY2
atan(3/14), 70, 0, 0 ... % FY3
];
river_ub = [ ...
pi, 140, 50, sqrt(2*1800/9.8), ... % FY1
2*pi-atan(14/80), 140, 50, sqrt(2*1400/9.8), ... % FY2
pi-atan(3/6), 140, 50, sqrt(2*700/9.8) ... % FY3
];

window_options = optimoptions('ga', ...
'PopulationSize', 2000, ...
'MaxGenerations', 50, ...
'FunctionTolerance', 1e-3, ...
'ConstraintTolerance', 1e-3, ...
'Display', 'iter', ...
'CrossoverFraction', 0.9, ...
'MutationFcn', {@mutationuniform, 0.1}, ...
'PlotFcn', {@gaplotbestf}, ...
'UseParallel', true);

window_options = optimoptions(window_options, 'PlotFcn', []);

%% 适应度函数
window_fitfun = @(x) window_fitness_function_Q4(x,
water_constants);
[window_x_optimal, window_fval] = ga(window_fitfun, river_nvars,
[], [], [], [], river_lb, river_ub, [], window_options);

%% 结果解析
river_theta_opts = window_x_optimal([1, 5, 9]);
river_v_opts = window_x_optimal([2, 6, 10]);
river_t_fly_opts = window_x_optimal([3, 7, 11]);
river_t_fuse_opts = window_x_optimal([4, 8, 12]);

```

```

river_P_FY_starts = water_constants.river_P_FY__0s;
water_P_drops = zeros(3, 3);
water_P_dets = zeros(3, 3);
river_T_individuals = zeros(1, 3);

for river_i = 1:3
    river_P_FY0 = river_P_FY_starts{river_i};
    river_theta = river_theta_opts(river_i);
    river_v = river_v_opts(river_i);
    river_t_fly = river_t_fly_opts(river_i);
    river_t_fuse = river_t_fuse_opts(river_i);

    river_v_FY_vec = river_v * [cos(river_theta); sin(river_theta);
0];

    % 计算投放点坐标
    water_P_drops(:, river_i) = river_P_FY0 + river_v_FY_vec *
river_t_fly;
    % 计算起爆点坐标
    water_P_dets(:, river_i) = water_P_drops(:, river_i) +
river_v_FY_vec * river_t_fuse + [0; 0; -0.5 * water_g] *
river_t_fuse^2;

    river_T_individuals(river_i) =
window_calculate_individual_shielding_time(river_i, window_x_optimal,
water_constants);
end

water_max_shielding_time = -window_fval;
fprintf('找到的最大有效遮蔽时长: %.3f 秒\n\n',
water_max_shielding_time);

for river_i = 1:3
    fprintf('无人机 FY%d\n', river_i);
    fprintf('飞行方向: %.2f 度\n',
rad2deg(river_theta_opts(river_i)));
    fprintf('飞行速度: %.2f m/s\n', river_v_opts(river_i));
    fprintf('投放前飞行时间: %.2f s\n', river_t_fly_opts(river_i));
    fprintf('烟幕弹引信时间: %.2f s\n', river_t_fuse_opts(river_i));
    fprintf('投放点 (X,Y,Z): (%.2f, %.2f, %.2f) m\n', water_P_drops(1,
river_i), water_P_drops(2, river_i), water_P_drops(3, river_i));
    fprintf('起爆点 (X,Y,Z): (%.2f, %.2f, %.2f) m\n', water_P_dets(1,
river_i), water_P_dets(2, river_i), water_P_dets(3, river_i));

```

```

        fprintf('[独立有效遮蔽时长]: %.3f 秒\n',
river_T_individuals(river_i));
        fprintf('\n');
    end

%% 函数
function window_neg_T_effective = window_fitness_function_Q4(x,
window_constants)
    % 适应度函数: 计算给定 12 个决策变量 x 时的联合遮蔽时长, 并返回其负值。
    river_theta_all = x([1, 5, 9]);
    river_v_all = x([2, 6, 10]);
    river_t_fly_all = x([3, 7, 11]);
    river_t_fuse_all = x([4, 8, 12]);
    river_t_det_all = river_t_fly_all + river_t_fuse_all;

    [river_t_det_sorted, river_order] = sort(river_t_det_all);

    river_dt = 0.05; % 时间步长
    river_t_sim_vec = river_t_det_sorted(1) : river_dt :
river_t_det_sorted(3) + 20;
    if isempty(river_t_sim_vec)
        window_neg_T_effective = 0; return;
    end
    river_P_M1_0 = window_constants.river_P_M1_0;
    river_v_M1_mag = window_constants.river_v_M1_mag;
    river_u_M1 = ([0;0;0] - river_P_M1_0) / norm([0;0;0] -
river_P_M1_0);
    river_v_M1 = river_v_M1_mag * river_u_M1;
    river_P_M1_t = @(t) river_P_M1_0 + river_v_M1 * t;

    water_g = window_constants.water_g;
    window_P_cloud_t_funcs = cell(1,3);

    for river_i = 1:3
        river_idx = river_order(river_i);
        river_P_FY_0_i = window_constants.river_P_FY_0s{river_idx};
        river_v_vec_i = river_v_all(river_idx) *
[cos(river_theta_all(river_idx)); sin(river_theta_all(river_idx));
0];
        river_t_fly_i = river_t_fly_all(river_idx);
        river_t_fuse_i = river_t_fuse_all(river_idx);
        river_t_det_i = river_t_det_all(river_idx);

        water_P_drop_i = river_P_FY_0_i + river_v_vec_i * river_t_fly_i;
    end
end

```



```

        water_P_det_i = water_P_drop_i + river_v_vec_i * river_t_fuse_i +
[0;0;-0.5*water_g]*river_t_fuse_i^2;

        window_P_cloud_t_funcs{river_i} = @(t) water_P_det_i + [0;0;-3] *
max(0, t - river_t_det_i);
        end

        % — 圆柱轮廓采样
        water_K_T = [-2.58591305193437; 193.495151524606;
9.56619290243231];
        window_num_key_points = size(water_K_T, 2);
        water_R_cloud = window_constants.water_R_cloud;

        window_shielded_steps = 0;
        for river_t_current = river_t_sim_vec
            river_P_M1_current = river_P_M1_t(river_t_current);

            window_is_cloud1_active = (river_t_current >=
river_t_det_sorted(1)) && (river_t_current < river_t_det_sorted(1) +
20);
            window_is_cloud2_active = (river_t_current >=
river_t_det_sorted(2)) && (river_t_current < river_t_det_sorted(2) +
20);
            window_is_cloud3_active = (river_t_current >=
river_t_det_sorted(3)) && (river_t_current < river_t_det_sorted(3) +
20);

            window_occluded_point_indices_total = [];
            if window_is_cloud1_active
                water_P_cloud1_current =
window_P_cloud_t_funcs{1}(river_t_current);
                window_indices1 =
window_get_occluded_indices_Q4(river_P_M1_current,
water_P_cloud1_current, water_K_T, water_R_cloud);
                window_occluded_point_indices_total =
union(window_occluded_point_indices_total, window_indices1);
            end
            if window_is_cloud2_active
                water_P_cloud2_current =
window_P_cloud_t_funcs{2}(river_t_current);
                window_indices2 =
window_get_occluded_indices_Q4(river_P_M1_current,
water_P_cloud2_current, water_K_T, water_R_cloud);

```

```

        window_occluded_point_indices_total =
union(window_occluded_point_indices_total, window_indices2);
    end
    if window_is_cloud3_active
        water_P_cloud3_current =
window_P_cloud_t_funcs{3}(river_t_current);
        window_indices3 =
window_get_occluded_indices_Q4(river_P_M1_current,
water_P_cloud3_current, water_K_T, water_R_cloud);
        window_occluded_point_indices_total =
union(window_occluded_point_indices_total, window_indices3);
    end
    if length(window_occluded_point_indices_total) ==
window_num_key_points
        window_shielded_steps = window_shielded_steps + 1;
    end
    end

    water_T_effective = window_shielded_steps * river_dt;
    window_neg_T_effective = -water_T_effective; % GA 求最小值, 故返回负
值
    end

    %% 函数: 严格口径
    function window_occluded_indices =
window_get_occluded_indices_Q4(river_P_M1, water_P_cloud, water_K_T,
water_R_cloud)
        river_V_axis = water_P_cloud - river_P_M1;
        river_dist_mc = norm(river_V_axis);

        if river_dist_mc <= water_R_cloud
            window_occluded_indices = 1:size(water_K_T, 2);
            return;
        end

        water_alpha = asin(water_R_cloud / river_dist_mc);
        window_occluded_indices = [];

        for river_k = 1:size(water_K_T, 2)
            river_W = water_K_T(:, river_k) - river_P_M1;
            river_cos_beta_k = dot(river_V_axis, river_W) / (river_dist_mc *
norm(river_W));
            river_beta_k = acos(max(min(river_cos_beta_k, 1), -1));
            if river_beta_k <= water_alpha

```

```

        window_occluded_indices = [window_occluded_indices,
river_k]; %%ok<AGROW>
    end
    end
    end

    %% 函数：单机独立有效时长
    function water_T_individual =
window_calculate_individual_shielding_time(river_uav_idx, x,
window_constants)
        river_theta = x( (river_uav_idx-1)*4 + 1 );
        river_v = x( (river_uav_idx-1)*4 + 2 );
        river_t_fly = x( (river_uav_idx-1)*4 + 3 );
        river_t_fuse= x( (river_uav_idx-1)*4 + 4 );
        river_t_det = river_t_fly + river_t_fuse;

        river_P_FY_0 = window_constants.river_P_FY__0s{river_uav_idx};

        river_dt = 0.05;
        river_sim_time_vec = river_t_det : river_dt : river_t_det + 20;
        if isempty(river_sim_time_vec), water_T_individual = 0; return;
    end

        river_P_M1_0 = window_constants.river_P_M1_0;
        river_v_M1_mag = window_constants.river_v_M1_mag;
        river_u_M1 = ([0;0;0] - river_P_M1_0) / norm([0;0;0] -
river_P_M1_0);
        river_v_M1 = river_v_M1_mag * river_u_M1;
        river_P_M1_t = @(t) river_P_M1_0 + river_v_M1 * t;

        river_v_FY_vec = river_v * [cos(river_theta); sin(river_theta);
0];
        water_P_drop = river_P_FY_0 + river_v_FY_vec * river_t_fly;
        water_P_det = water_P_drop + river_v_FY_vec * river_t_fuse +
[0;0;-0.5*window_constants.water_g]*river_t_fuse^2;
        window_P_cloud_t = @(t) water_P_det + [0;0;-3] * max(0, t -
river_t_det);
        % 圆柱轮廓采样
        water_K_T = [-2.58591305193437; 193.495151524606;
9.56619290243231];
        window_num_key_points = size(water_K_T, 2);

        window_shielded_steps = 0;
        for river_t_current = river_sim_time_vec
            river_P_M1_current = river_P_M1_t(river_t_current);

```

```

        water_P_cloud_current = window_P_cloud_t(river_t_current);
        window_occluded_indices =
window_get_occluded_indices_Q4(river_P_M1_current,
water_P_cloud_current, water_K_T, window_constants.water_R_cloud);
        if length(window_occluded_indices) == window_num_key_points
            window_shielded_steps = window_shielded_steps + 1;
        end
    end
    water_T_individual = window_shielded_steps * river_dt;
end

```

附录 5

利用：matlab 求解第五代码

```

clc; clear; close all;
motherboard.keyboard_g = 9.8;

% 三枚导弹初始位置信息
motherboard.gpu_P_M_0s = {
[20000; 0; 2000], ... % M1 导弹
[19000; 600; 2100], ... % M2 导弹
[18000; -600; 1900] ... % M3 导弹
};
motherboard.cpu_v_M_mag = 300;

% 五架无人机初始位置信息
motherboard.jacket_P_FY_0s = {
[17800; 0; 1800], ... % FY1
[12000; 1400; 1400], ... % FY2
[6000; -3000; 700], ... % FY3
[11000; 2000; 1800], ... % FY4
[13000; -2000; 1300] ... % FY5
};

motherboard.ram_R_T = 7; % 真目标半径
motherboard.hdd_H_T = 10; % 真目标高度
motherboard.monitor_C_T = [0; 200; 0]; % 真目标底面圆心
motherboard.router_R_cloud = 10; % 云团半径
motherboard.scarf_T_smoke_effective = 20; % 烟幕有效时长

% 圆柱关键点采样（上/中/下三圈，各取 sock_circle_points 个点）
sock_circle_points = 10;

```

```

        hat_circle_theta = linspace(0, 2*pi, sock_circle_points+1);
    hat_circle_theta(end) = [];
        shirt_circle_x = motherboard.monitor_C_T(1) + motherboard.ram_R_T
    * cos(hat_circle_theta);
        pants_circle_y = motherboard.monitor_C_T(2) + motherboard.ram_R_T
    * sin(hat_circle_theta);
        shoe_z_bottom = motherboard.monitor_C_T(3) * ones(1,
    sock_circle_points);
        shoe_z_middle = (motherboard.monitor_C_T(3) +
    motherboard.hdd_H_T/2) * ones(1, sock_circle_points);
        shoe_z_top = (motherboard.monitor_C_T(3) + motherboard.hdd_H_T) *
    ones(1, sock_circle_points);
        keyboard_K_T_bottom = [shirt_circle_x; pants_circle_y;
    shoe_z_bottom];
        keyboard_K_T_middle = [shirt_circle_x; pants_circle_y;
    shoe_z_middle];
        keyboard_K_T_top = [shirt_circle_x; pants_circle_y; shoe_z_top];
        motherboard.keyboard_K_T = [keyboard_K_T_bottom,
    keyboard_K_T_middle, keyboard_K_T_top];
        motherboard.mouse_num_key_points = size(motherboard.keyboard_K_T,
    2);

    % 三枚导弹的运动函数（恒速直线指向假目标原点）
    webcam_Fake_Target = [0; 0; 0];
    motherboard.psu_P_M_t_funcs = cell(1, 3);
    for i = 1:3
        cpu_u_M = (webcam_Fake_Target - motherboard.gpu_P_M_0s{i}) /
    norm(webcam_Fake_Target - motherboard.gpu_P_M_0s{i});
        cpu_v_M = motherboard.cpu_v_M_mag * cpu_u_M;
        motherboard.psu_P_M_t_funcs{i} = @(t) motherboard.gpu_P_M_0s{i} +
    cpu_v_M * t;
    end

    %% 先分别优化每架无人机（独立优化） %%
    coat_num_uavs = 5;
    shirt_x_individuals = cell(1, coat_num_uavs);
    shirt_fval_individuals = zeros(1, coat_num_uavs);

    cpu_nvars_single = 8;
    cpu_Aineq_single = [0 0 1 0 -1 0 0 0;
    0 0 0 0 1 0 -1 0];
    cpu_bineq_single = [-1; -1];

    for i = 1:coat_num_uavs

```

```

fprintf('正在优化无人机 FY%d\n', i);
p = motherboard.jacket_P_FY_0s{i}; x_F = p(1); y_F = p(2); z_F =
p(3);
rt = sqrt(z_F*2/9.8);

switch i
case 1
a1 = 0;
a2 = 2*pi - atan(200/600);
case {2,4}
a1 = pi + atan((y_F-200)/x_F);
a2 = 2*pi - atan((y_F-600)/(19000-x_F));
case {3,5}
a1 = atan((-y_F-600)/(18000-x_F));
a2 = pi - atan(-y_F/x_F);
end

jacket_lb = [a1, 70, 0, 0, 0, 0, 0, 0];
jacket_ub = [a2, 140, inf, rt, inf, rt, inf, rt];

sock_pop_size = 500;
sock_max_gen = 100;
end

keyboard_options = optimoptions('ga', ...
'PopulationSize', sock_pop_size, ...
'MaxGenerations', sock_max_gen, ...
'FunctionTolerance', 1e-4, ...
'ConstraintTolerance', 1e-4, ...
'Display', 'iter', ...
'PlotFcn', [], ...
'UseParallel', true);

keyboard_fitfun = @(x) keyboard_fitness_function_Q5_single(x, i,
motherboard);
[keyboard_x_opt, keyboard_fval] = ga(keyboard_fitfun,
cpu_nvars_single, cpu_Aineq_single, cpu_bineq_single, [], [],
jacket_lb, jacket_ub, [], keyboard_options);

shirt_x_individuals{i} = keyboard_x_opt;
shirt_fval_individuals(i) = keyboard_fval;

fprintf('FY%d 的最优独立策略参数:\n', i);

```

```

disp(keyboard_x_opt);
fprintf('FY%d 对三枚导弹形成的最优的独立遮蔽总时长: %.3f s\n\n', i, -
keyboard_fval);
end

%%协同优化%%
scarf_initial_state = cell2mat(shirt_x_individuals);
cpu_nvars_total = cpu_nvars_single * coat_num_uavs;

jacket_lb_total = []; jacket_ub_total = [];
for i = 1:coat_num_uavs
    jacket_P_FY_0_i = motherboard.jacket_P_FY_0s{i};
    x_F = jacket_P_FY_0_i(1); y_F = jacket_P_FY_0_i(2); z_F =
jacket_P_FY_0_i(3);
    switch i
    case 1
        jacket_lb_i = [0, 70, 0, 0, 0, 0, 0, 0];
        jacket_ub_i = [2*pi - atan(200/600), 140, 50, sqrt(z_F*2/9.8), 50,
sqrt(z_F*2/9.8), 50, sqrt(z_F*2/9.8)];
    case 2
        jacket_lb_i = [pi + atan((y_F-200)/x_F), 70, 0, 0, 0, 0, 0, 0];
        jacket_ub_i = [2*pi - atan((y_F-600)/(19000-x_F)), 140, 50,
sqrt(z_F*2/9.8), 50, sqrt(z_F*2/9.8), 50, sqrt(z_F*2/9.8)];
    case 3
        jacket_lb_i = [atan((-y_F-600)/(18000-x_F)), 70, 0, 0, 0, 0, 0,
0];
        jacket_ub_i = [pi - atan(-y_F/x_F), 140, 50, sqrt(z_F*2/9.8), 50,
sqrt(z_F*2/9.8), 50, sqrt(z_F*2/9.8)];
    case 4
        jacket_lb_i = [pi + atan((y_F-200)/x_F), 70, 0, 0, 0, 0, 0, 0];
        jacket_ub_i = [2*pi - atan((y_F-600)/(19000-x_F)), 140, 50,
sqrt(z_F*2/9.8), 50, sqrt(z_F*2/9.8), 50, sqrt(z_F*2/9.8)];
    case 5
        jacket_lb_i = [atan((-y_F-600)/(18000-x_F)), 70, 0, 0, 0, 0, 0,
0];
        jacket_ub_i = [pi - atan(-y_F/x_F), 140, 50, sqrt(z_F*2/9.8), 50,
sqrt(z_F*2/9.8), 50, sqrt(z_F*2/9.8)];
    end
    jacket_lb_total = [jacket_lb_total, jacket_lb_i];
    jacket_ub_total = [jacket_ub_total, jacket_ub_i];
end

cpu_Aineq_total = kron(eye(coat_num_uavs), cpu_Aineq_single);
cpu_bineq_total = kron(ones(coat_num_uavs,1), cpu_bineq_single);

```

```

keyboard_options_total = optimoptions('ga', ...
    'PopulationSize', 500, ...
    'MaxGenerations', 100, ...
    'FunctionTolerance', 1e-4, ...
    'ConstraintTolerance', 1e-4, ...
    'InitialPopulationMatrix', scarf_initial_state, ...
    'Display', 'iter', ...
    'PlotFcn', [], ...
    'UseParallel', true);

keyboard_fitfun_total = @(x)
keyboard_fitness_function_Q5_collective(x, motherboard);
[keyboard_x_all_optimal, keyboard_fval_all] =
ga(keyboard_fitfun_total, cpu_nvars_total, cpu_Aineq_total,
cpu_bineq_total, [], [], jacket_lb_total, jacket_ub_total, [],
keyboard_options_total);

scarf_max_total_shielding_time = -keyboard_fval_all;
fprintf('最大总有效遮蔽时长为: %.3f 秒\n\n',
scarf_max_total_shielding_time);

for i = 1:coat_num_uavs
    shirt_x_uav = keyboard_x_all_optimal((i-1)*cpu_nvars_single + 1 :
i*cpu_nvars_single);
    jacket_P_FY0 = motherboard.jacket_P_FY_0s{i};
    keyboard_g_local = motherboard.keyboard_g;

    hat_theta_opt = shirt_x_uav(1);
    hat_v_opt = shirt_x_uav(2);
    glove_t_fly_opts = [shirt_x_uav(3), shirt_x_uav(5),
shirt_x_uav(7)];
    glove_t_fuse_opts = [shirt_x_uav(4), shirt_x_uav(6),
shirt_x_uav(8)];

    jacket_v_FY_vec = hat_v_opt * [cos(hat_theta_opt);
sin(hat_theta_opt); 0];

    fprintf('无人机 FY%d 最优协同策略\n', i);
    fprintf('飞行方向: %.3f 度\n', rad2deg(hat_theta_opt));
    fprintf('飞行速度: %.3f m/s\n', hat_v_opt);

    for k = 1:3

```



```

        monitor_P_drop = jacket_P_FY0 + jacket_v_FY_vec *
glove_t_fly_opts(k);
        monitor_P_det = monitor_P_drop + jacket_v_FY_vec *
glove_t_fuse_opts(k) + [0;0;-
0.5*keyboard_g_local]*glove_t_fuse_opts(k)^2;

        fprintf('第 %d 枚烟幕弹:\n', k);
        fprintf('投放前飞行时间: %.3f s\n', glove_t_fly_opts(k));
        fprintf('烟幕弹引信时间: %.3f s\n', glove_t_fuse_opts(k));
        fprintf('投放点 (X,Y,Z): (%.2f, %.2f, %.2f) m\n',
monitor_P_drop(1), monitor_P_drop(2), monitor_P_drop(3));
        fprintf('起爆点 (X,Y,Z): (%.2f, %.2f, %.2f) m\n',
monitor_P_det(1), monitor_P_det(2), monitor_P_det(3));

        [~, belt_contribution_str] =
coat_analyze_individual_bomb_contribution(i, k,
keyboard_x_all_optimal, motherboard);
        fprintf('%s\n', belt_contribution_str);
    end
    fprintf('\n');
end

% 函数区
function neg_T_total = keyboard_fitness_function_Q5_single(x,
hoodie_uav_idx, motherboard)
    scarf_theta_FY = x(1);
    scarf_v_FY = x(2);
    glove_t_fly_all = [x(3), x(5), x(7)];
    glove_t_fuse_all = [x(4), x(6), x(8)];
    jacket_P_FY0 = motherboard.jacket_P_FY_0s{hoodie_uav_idx};
    jacket_v_FY_vec = scarf_v_FY * [cos(scarf_theta_FY);
sin(scarf_theta_FY); 0];

    cpu_t_det_unsorted = glove_t_fly_all + glove_t_fuse_all;
    [cpu_t_det_sorted, cpu_order] = sort(cpu_t_det_unsorted);
    glove_t_fly_sorted = glove_t_fly_all(cpu_order);

    if cpu_t_det_sorted(1) > 100
        neg_T_total = 0; return;
    end

    router_P_cloud_t_funcs = cell(1,3);
    for k = 1:3
        glove_t_fly_k = glove_t_fly_sorted(k);

```

```

        cpu_t_det_k = cpu_t_det_sorted(k);
        monitor_P_drop_k = jacket_P_FY0 + jacket_v_FY_vec * glove_t_fly_k;
        glove_t_fuse_k = cpu_t_det_k - glove_t_fly_k;
        monitor_P_det_pos_k = monitor_P_drop_k + jacket_v_FY_vec *
glove_t_fuse_k + [0;0;-0.5*motherboard.keyboard_g]*glove_t_fuse_k^2;
        router_P_cloud_t_funcs{k} = @(t) monitor_P_det_pos_k + [0;0;-3] *
max(0, t - cpu_t_det_k);
    end

    cpu_dt_sim = 0.05;
    cpu_t_sim_start = cpu_t_det_sorted(1);
    cpu_t_sim_end = cpu_t_det_sorted(3) +
motherboard.scarf_T_smoke_effective;
    if cpu_t_sim_start >= cpu_t_sim_end
        neg_T_total = 0; return;
    end
    cpu_sim_time_vec = cpu_t_sim_start : cpu_dt_sim : cpu_t_sim_end;

    shirt_T_shielding_total_for_all_missiles = 0;
    for m = 1:3
        psu_P_M_t = motherboard.psu_P_M_t_funcs{m};
        coat_shielded_steps = 0;
        for cpu_t_current = cpu_sim_time_vec
            gpu_P_M_current = psu_P_M_t(cpu_t_current);
            chassis_occluded_points_union = [];

            if cpu_t_current >= cpu_t_det_sorted(1) && cpu_t_current <
cpu_t_det_sorted(1) + motherboard.scarf_T_smoke_effective
                router_P_cloud1_current =
router_P_cloud_t_funcs{1}(cpu_t_current);
                mouse_indices1 = mouse_get_occluded_indices(gpu_P_M_current,
router_P_cloud1_current, motherboard);
                chassis_occluded_points_union =
union(chassis_occluded_points_union, mouse_indices1);
            end
            if cpu_t_current >= cpu_t_det_sorted(2) && cpu_t_current <
cpu_t_det_sorted(2) + motherboard.scarf_T_smoke_effective
                router_P_cloud2_current =
router_P_cloud_t_funcs{2}(cpu_t_current);
                mouse_indices2 = mouse_get_occluded_indices(gpu_P_M_current,
router_P_cloud2_current, motherboard);
                chassis_occluded_points_union =
union(chassis_occluded_points_union, mouse_indices2);
            end
        end
    end

```

```

        if cpu_t_current >= cpu_t_det_sorted(3) && cpu_t_current <
cpu_t_det_sorted(3) + motherboard.scarf_T_smoke_effective
            router_P_cloud3_current =
router_P_cloud_t_funcs{3}(cpu_t_current);
            mouse_indices3 = mouse_get_occluded_indices(gpu_P_M_current,
router_P_cloud3_current, motherboard);
            chassis_occluded_points_union =
union(chassis_occluded_points_union, mouse_indices3);
        end

        if length(chassis_occluded_points_union) ==
motherboard.mouse_num_key_points
            coat_shielded_steps = coat_shielded_steps + 1;
        end
    end

    shirt_T_shielding_total_for_all_missiles =
shirt_T_shielding_total_for_all_missiles + coat_shielded_steps *
cpu_dt_sim;
    end

    neg_T_total = -shirt_T_shielding_total_for_all_missiles;
    end

function neg_T_total = keyboard_fitness_function_Q5_collective(x,
motherboard)
    coat_num_uavs = 5; cpu_nvars_single = 8;
    hat_all_bombs_info = struct([]); hat_bomb_counter = 0;

    for i = 1:coat_num_uavs
        shirt_x_uav = x((i-1)*cpu_nvars_single + 1 : i*cpu_nvars_single);
        scarf_theta_FY = shirt_x_uav(1);
        scarf_v_FY = shirt_x_uav(2);
        glove_t_fly_all = [shirt_x_uav(3), shirt_x_uav(5),
shirt_x_uav(7)];
        glove_t_fuse_all = [shirt_x_uav(4), shirt_x_uav(6),
shirt_x_uav(8)];

        jacket_P_FY0 = motherboard.jacket_P_FY_0s{i};
        jacket_v_FY_vec = scarf_v_FY * [cos(scarf_theta_FY);
sin(scarf_theta_FY); 0];

        for k = 1:3
            hat_bomb_counter = hat_bomb_counter + 1;
            glove_t_fly = glove_t_fly_all(k);
            glove_t_fuse = glove_t_fuse_all(k);

```

```

cpu_t_det = glove_t_fly + glove_t_fuse;

monitor_P_drop = jacket_P_FY0 + jacket_v_FY_vec * glove_t_fly;
monitor_P_det_pos = monitor_P_drop + jacket_v_FY_vec *
glove_t_fuse + [0;0;-0.5*motherboard.keyboard_g]*glove_t_fuse^2;

hat_all_bombs_info(hat_bomb_counter).tie_t_det = cpu_t_det;
hat_all_bombs_info(hat_bomb_counter).monitor_P_det_pos =
monitor_P_det_pos;
end
end

cpu_all_t_dets = [hat_all_bombs_info.tie_t_det];
cpu_dt_sim = 0.05;
cpu_t_sim_start = min(cpu_all_t_dets);
cpu_t_sim_end = max(cpu_all_t_dets) +
motherboard.scarf_T_smoke_effective;
if cpu_t_sim_start >= cpu_t_sim_end || cpu_t_sim_start > 100
neg_T_total = 0; return;
end
cpu_sim_time_vec = cpu_t_sim_start : cpu_dt_sim : cpu_t_sim_end;

shirt_T_shielding_total_for_all_missiles = 0;
for m = 1:3
psu_P_M_t = motherboard.psu_P_M_t_funcs{m};
coat_shielded_steps = 0;

for cpu_t_current = cpu_sim_time_vec
gpu_P_M_current = psu_P_M_t(cpu_t_current);
chassis_occluded_points_union = [];

for j = 1:length(hat_all_bombs_info)
tie_bomb = hat_all_bombs_info(j);
if cpu_t_current >= tie_bomb.tie_t_det && cpu_t_current <
tie_bomb.tie_t_det + motherboard.scarf_T_smoke_effective
router_P_cloud_current = tie_bomb.monitor_P_det_pos + [0;0;-3] *
(cpu_t_current - tie_bomb.tie_t_det);
mouse_indices = mouse_get_occluded_indices(gpu_P_M_current,
router_P_cloud_current, motherboard);
chassis_occluded_points_union =
union(chassis_occluded_points_union, mouse_indices);
if length(chassis_occluded_points_union) ==
motherboard.mouse_num_key_points
break;

```

```

end
end
end

if length(chassis_occluded_points_union) ==
motherboard.mouse_num_key_points
    coat_shielded_steps = coat_shielded_steps + 1;
end
end
shirt_T_shielding_total_for_all_missiles =
shirt_T_shielding_total_for_all_missiles + coat_shielded_steps *
cpu_dt_sim;
end
neg_T_total = -shirt_T_shielding_total_for_all_missiles;
end

function mouse_indices = mouse_get_occluded_indices(gpu_P_M,
router_P_cloud, motherboard)
    keyboard_K_T = motherboard.keyboard_K_T; router_R_cloud =
motherboard.router_R_cloud;
    gpu_V_axis = router_P_cloud - gpu_P_M;
    cpu_dist_missile_cloud = norm(gpu_V_axis);
    if cpu_dist_missile_cloud <= router_R_cloud
        mouse_indices = 1:motherboard.mouse_num_key_points; return;
    end
    cpu_alpha = asin(router_R_cloud / cpu_dist_missile_cloud);
    mouse_indices = [];
    for k = 1:motherboard.mouse_num_key_points
        window_W = keyboard_K_T(:, k) - gpu_P_M;
        cpu_cos_beta_k = dot(gpu_V_axis, window_W) /
(cpu_dist_missile_cloud * norm(window_W));
        cpu_beta_k = acos(max(min(cpu_cos_beta_k, 1), -1));
        if cpu_beta_k <= cpu_alpha, mouse_indices = [mouse_indices, k];
    end
end
end

function [shirt_T_total_effective, belt_contribution_str] =
coat_analyze_individual_bomb_contribution(shirt_uav_idx,
hat_bomb_idx_in_uav, keyboard_x_all_optimal, motherboard)
    cpu_nvars_single = 8;
    shirt_x_uav = keyboard_x_all_optimal((shirt_uav_idx-
1)*cpu_nvars_single + 1 : shirt_uav_idx*cpu_nvars_single);
    scarf_theta_FY = shirt_x_uav(1); scarf_v_FY = shirt_x_uav(2);

```

```

glove_t_fly = shirt_x_uav(2*hat_bomb_idx_in_uav + 1);
glove_t_fuse = shirt_x_uav(2*hat_bomb_idx_in_uav + 2);
cpu_t_det = glove_t_fly + glove_t_fuse;

jacket_P_FY0 = motherboard.jacket_P_FY_0s{shirt_uav_idx};
jacket_v_FY_vec = scarf_v_FY * [cos(scarf_theta_FY);
sin(scarf_theta_FY); 0];

monitor_P_drop = jacket_P_FY0 + jacket_v_FY_vec * glove_t_fly;
monitor_P_det_pos = monitor_P_drop + jacket_v_FY_vec *
glove_t_fuse + [0;0;-0.5*motherboard.keyboard_g]*glove_t_fuse^2;
router_P_cloud_t = @(t) monitor_P_det_pos + [0;0;-3] * max(0, t -
cpu_t_det);

cpu_dt_sim = 0.05;
cpu_sim_time_vec = cpu_t_det : cpu_dt_sim : cpu_t_det +
motherboard.scarf_T_smoke_effective;

shirt_T_effective_per_missile = zeros(1,3);
for m = 1:3
psu_P_M_t = motherboard.psu_P_M_t_funcs{m};
coat_shielded_steps = 0;
for cpu_t_current = cpu_sim_time_vec
gpu_P_M_current = psu_P_M_t(cpu_t_current);
router_P_cloud_cur = router_P_cloud_t(cpu_t_current);
mouse_indices = mouse_get_occluded_indices(gpu_P_M_current,
router_P_cloud_cur, motherboard);
if length(mouse_indices) == motherboard.mouse_num_key_points
coat_shielded_steps = coat_shielded_steps + 1;
end
end
shirt_T_effective_per_missile(m) = coat_shielded_steps *
cpu_dt_sim;
end

shirt_T_total_effective = sum(shirt_T_effective_per_missile);
tie_parts = {};
if shirt_T_effective_per_missile(1) > 0.01, tie_parts{end+1} =
sprintf('M1: %.2fs', shirt_T_effective_per_missile(1)); end
if shirt_T_effective_per_missile(2) > 0.01, tie_parts{end+1} =
sprintf('M2: %.2fs', shirt_T_effective_per_missile(2)); end
if shirt_T_effective_per_missile(3) > 0.01, tie_parts{end+1} =
sprintf('M3: %.2fs', shirt_T_effective_per_missile(3)); end
belt_contribution_str = '单弹有效干扰时长: ';

```

```
    if isempty(tie_parts)
        belt_contribution_str = [belt_contribution_str '0.00s'];
    else
        belt_contribution_str = [belt_contribution_str strjoin(tie_parts,
            ', ')];
    end
end
```